

Event Ticket Booking with QR Code Validation

*A Project Report Submitted in partial fulfillment of the requirements
for the award of the degree of*

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

Submitted By

S.RAVI	22221A05A1
N.H.S.SRIRAM	22221A0580
S.S.S.PRAVEEN	22221A05B0
P.VEERANNA	22221A0587
CH.S.KOPPESWARARAO	23225A0502

Under the Esteemed Guidance of

Mr. T.V.L.SRINIVASA RAO

M.Tech.

Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
BONAM VENKATA CHALAMAYYA ENGINEERING COLLEGE
(AUTONOMOUS)

(Approved by A.I.C.T.E, New Delhi & Affiliated to J.N.T.U.K, Kakinada)

(Accredited by NBA & NAAC with "A" Grade)

ODALAREVU-533210

2022 –2026

BONAM VENKATA CHALAMAYYA ENGINEERING COLLEGE
(AUTONOMOUS)
ODALAREVU-533210

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the project work entitled “**Event Ticket Booking with QR Code Validation**” is being submitted for the partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering (CSE)** to **Bonam Venkata Chalamayya Engineering College (A), Odalarevu**. This is a bonafide work done by **S.Ravi(22221A05A1), N.H.S.Sriram(22221A0580), S.S.S.Praveen(22221A05B0), P.Veeranna(22221A0587) and Ch.S.Koppeswararao(23225A0502)** under my guidance during the academic year 2022–2026, and it has been found suitable for acceptance according to the requirements of the University.

The results embodied in this project have not been submitted to any other University or Institute for the award of any degree.

Project Guide
Mr. T.V.L.SRINIVASA RAO
M.Tech.,
AssistantProfessor

Head of the Department
Dr. MEERASHARIF SHEIK
M.Tech., Ph.D,
Professor & HOD

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We are profoundly grateful to express our deep sense of gratitude and respect towards our guide **Mr. T.V.L.SRINIVASA RAO , M.Tech.,Assistant Professor** of CSE Department, Bonam Venkata Chalamayya Engineering College (A), Odalarevu, for his excellent guidance right from selection of the project and his valuable suggestions throughout the project work. His constant encouragement and support has been the cause of our success in completing this thesis in the college.

We are also grateful to the Project Co-Ordinator **Mr.M.V.K.SUBASH, M.Tech., (Ph.D.), Assistant Professor** of CSE Department, Bonam Venkata Chalamayya Engineering College (A), Odalarevu, for his excellent guidance right from selection of the project and his valuable suggestions throughout the project work.

We really thankful to **Dr.MEERASHARIF SHEIK, M.Tech., Ph.D., Professor and Head of the Department for Computer Science and Engineering** for providing the laboratory facilities to the fullest extent as and when required and also for giving us the opportunity to carry out the project work in the college.

We also thankful to **Dr.MAHESWAR DUTTA, M.E., Ph.D., Principal, Bonam Venkata Chalamayya Engineering** College (A), Odalarevu, for his support during and till the completion of the project.

We are also thankful to our **MANAGEMENT**, Bonam Venkata Chalamayya Engineering College (A), Odalarevu, for their support during and till the completion of the project.

We are thankful to all the **TEACHING AND NON-TEACHING STAFF** of Computer Science and Engineering Department, Bonam Venkata Chalamayya Engineering College (A), Odalarevu, and **FRIENDS** for their direct and indirect help provided to us in completing the project.

Last but not least, we own all our success to our **FAMILY MEMBERS, CLASS MATES AND TEACHERS** from our childhood, whose vision, love and inspiration made us to reach out for these glories.

Project Associates.....

S.RAVI	22221A05A1
N.H.S.SRIRAM	22221A0580
S.S.S.PRAVEEN	22221A05B0
P.VEERANNA	22221A0587
CH.S.KOPPESWARAO	23225A0502

ABSTRACT

Event Ticket Booking with QR Code Validation is a complete web-based application designed to provide an efficient and user-friendly platform for booking and managing event tickets online. This system allows users to view available events, book tickets, and receive a **unique QR code** for entry verification. The application is developed using modern web technologies such as **React.js for the frontend, Spring Boot for the backend, and MySQL for database management.**

The system consists of two main modules: User Module and Admin Module. The User Module allows users to register, log in, browse events, book tickets, and view their booked tickets in the form of QR codes. The Admin Module enables administrators to add and manage events, monitor ticket bookings, and validate tickets by scanning QR codes at the event entry point.

Each ticket generated by the system contains a unique QR code which is used for secure verification during event entry. When the QR code is scanned, the system checks the ticket's validity and updates its status to prevent duplicate usage. This ensures a high level of security and prevents ticket fraud.

The Event Ticket Booking System automates the entire ticketing process, reducing manual work and improving accuracy. It provides a smooth and reliable experience for both users and event organizers. By using this system, events can be managed more efficiently, customer experience is enhanced, and ticket validation becomes faster and more secure.

This project demonstrates the effective use of **full-stack web development** to build a real world application that supports online booking, digital tickets, and secure event management.

In addition, the system is designed with a modern and responsive user interface, making it accessible on both desktop and mobile devices. The application ensures fast communication between the frontend and backend through **RESTful APIs**, providing real-time updates for ticket booking and validation. Proper authentication and role-based access control are implemented to maintain data security between users and administrators. Overall, this project provides a scalable and reliable solution for managing event ticketing in a digital and automated manner.

TABLE OF CONTENTS

ABSTRACT.....	i
TABLE OF CONTENTS.....	ii-iv
LIST OF FIGURES.....	v
CHAPTER – 1.....	6-7
INTRODUCTION.....	6-7
CHAPTER – 2.....	8-9
LITERATURE SURVEY.....	8-9
CHAPTER – 3.....	10-11
SYSTEM STUDY / FEASIBILITY STUDY.....	10-11
CHAPTER – 4.....	12-13
SYSTEM REQUIREMENTS SPECIFICATIONS.....	12
4.1 Existing System.....	12
4.2 Proposed System.....	12
4.3 Hardware & Software Requirements.....	13
CHAPTER – 5.....	14-21
SYSTEM ARCHITECTURE.....	14
UML Diagrams.....	15
5.1 Use Case Diagram.....	16
5.2 Class Diagram.....	17
5.3 Sequence Diagram.....	18
5.4 Collaboration Diagram.....	19
5.5 Data Flow Diagram.....	20
CHAPTER – 6.....	22-23
IMPLEMENTATION.....	22
6.1 Modules.....	22-23

CHAPTER – 7.....	24-28
SOFTWARE ENVIRONMENT.....	24
7.1 Introduction to Software Environment.....	24
7.1.1 Operating Systems.....	24
7.2 Java Programming Language.....	24
7.2.1 Overview of Java.....	24-25
7.2.2 Key Features of Java.....	25
7.3 Spring Boot Framework.....	25
7.3.1 Introduction to Spring Boot.....	25
7.3.2 Role of Spring Boot in the Project.....	25
7.3.3 Spring MVC.....	26
7.3.4 Advantages of Spring Boot.....	26
7.4 Frontend Technologies.....	26
7.4.1 React Framework.....	26
7.4.2 TypeScript and JSX.....	26
7.4.3 CSS for UI Design.....	27
7.4.4 Advantages of React Framework.....	27
7.5 Backend Technologies.....	27
7.5.1 RESTful Web Services.....	27
7.5.2 Spring Data JPA.....	27
7.5.3 Hibernate ORM.....	27-28
7.6 Database Management System.....	28
7.6.1 MySQL Database.....	28
CHAPTER – 8.....	29-48
STRUCTURE.....	29-31
SAMPLE CODE.....	32-48
CHAPTER – 9.....	49-50
9.1 Unit Testing.....	49
9.2 Integration Testing.....	49-50

9.3 Functional Testing.....	50
9.4 Acceptance Testing.....	50
CHAPTER – 10.....	51-57
RESULTS AND OUTPUT SCREENS.....	51-57
CHAPTER – 11.....	58
CONCLUSION.....	58
CHAPTER – 12.....	59-60
REFERENCES.....	59-60

LIST OF FIGURES

Fig 5.1 System Architecture.....	14
Fig 5.2 Use Case Diagram.....	16
Fig 5.3 Class Diagram.....	17
Fig 5.4 Sequence Diagram.....	18
Fig 5.5 Collaboration Diagram.....	19
Fig 5.6 Data Flow Diagram.....	20
Fig 5.7 Deployment Diagram.....	21
Fig 10.1 Home Page.....	51
Fig 10.2 User Login Page.....	52
Fig 10.3 User Registration Page.....	52
Fig 10.4 Admin Dashboard.....	53
Fig 10.5 Available Events.....	53
Fig 10.6 Scan QR Code.....	54
Fig 10.7 Create New Event.....	54
Fig 10.8 Event Management.....	55
Fig 10.9 Events Display Screen.....	55
Fig 10.10 Seats.....	56
Fig 10.11 User and Admin Profile.....	56
Fig 10.12 Payment Section.....	57
Fig 10.13 Tickets Section.....	57

CHAPTER – 1

INTRODUCTION

In the modern digital era, technology has become an essential part of everyday life. Almost all activities such as shopping, education, banking, entertainment, and travel are now handled through online platforms. One of the areas that has significantly benefited from digital transformation is event management and ticket booking. Earlier, event tickets were sold manually through physical counters or agents, which often resulted in long queues, ticket duplication, and inconvenience to customers. With the growth of the internet and web applications, online ticket booking systems have emerged as an effective solution to these problems. The Event Ticket Booking System is a web-based application developed to provide a simple, fast, and secure platform for managing events and booking tickets online. This system allows users to browse available events, book tickets, and receive a digital ticket in the form of a QR code. Event organizers or administrators can add new events, manage ticket availability, and verify tickets using

QR code scanning at the event entrance. This digital approach makes the entire ticketing process more efficient, transparent, and reliable.

In traditional ticketing systems, customers were required to visit a physical location to buy tickets. This method was time-consuming and inconvenient, especially for people living far away or for events that were very popular. Moreover, manual ticketing often led to issues such as lost tickets, forged tickets, and improper record keeping. There was also no easy way to verify whether a ticket was genuine or already used. These problems created the need for a digital solution that could handle ticket booking and validation in a more secure and organized manner.

The Event Ticket Booking System solves these problems by introducing online booking and QR code-based ticket validation. Each ticket generated by the system contains a unique QR code, which

can be scanned at the event entry point. When the QR code is scanned, the system checks the ticket

details in the database and confirms whether it is valid, already used, or invalid.

Another important feature of this system is the separation of roles between users and administrators. Users can create an account, log in, view upcoming events, book tickets, and see their booked tickets. They can also view the QR codes associated with their tickets. On the other hand, administrators have special privileges such as adding new events, updating event details, viewing ticket bookings, and validating tickets. This role-based access ensures proper control and security within the system.

The system is designed using modern web technologies to provide a smooth and responsive user experience. The frontend is developed using React, which helps in creating interactive and user friendly web pages. The backend is developed using Spring Boot, which provides a powerful and secure framework for handling business logic, database operations, and API communication. The MySQL database is used to store information related to users, events, tickets, and QR codes. The combination of these technologies ensures that the system is reliable, scalable, and easy to maintain.

One of the main objectives of this project is to reduce the complexity involved in event ticket booking. With this system, users do not need to carry physical tickets, as all ticket information is stored digitally and can be accessed anytime. The QR code system allows quick and error-free ticket verification, which reduces waiting time at event entrances and improves the overall event experience. It also helps event organizers to manage large crowds efficiently.

Security is another important aspect of this project. The system ensures that each user has a unique login account, and all ticket data is securely stored in the database. The QR code validation prevents multiple uses of the same ticket, thereby avoiding misuse. Only authorized administrators can access event management and ticket validation features, which further improves system security.

In addition to providing convenience to users, the Event Ticket Booking System also helps event organizers by providing a centralized platform for managing all event-related data. Organizers can easily track ticket sales, monitor attendance, and generate reports. This data can be used to improve future events and marketing strategies.

CHAPTER - 2

LITERATURE SURVEY

The literature survey provides an overview of existing ticket booking systems and related technologies that are used for managing events and ticketing. It helps in understanding the advantages and limitations of current systems and how the proposed Event Ticket Booking System improves upon them. This chapter is divided into three modules based on different types of ticketing systems.

Module – A : Manual Ticket Booking Systems

In earlier days, event ticket booking was done manually using physical tickets. Customers had to visit a ticket counter or an authorized agent to purchase tickets. This process was slow and inconvenient, especially when there was high demand for an event. People had to stand in long queues, which wasted time and caused frustration. In many cases, tickets would sell out quickly, and some customers would not get a chance to buy them.

Manual ticketing systems also had many management problems. Event organizers had to maintain paper records of ticket sales, which was difficult and prone to errors. It was hard to keep track of how many tickets were sold and how many were remaining. Sometimes tickets were printed more than required, leading to confusion and loss. There was also a high chance of tickets being lost, damaged, or stolen.

Another major problem with manual ticketing was ticket verification. At the event entrance, staff members had to manually check each ticket. This made the entry process slow and inefficient. It was also difficult to identify fake or duplicate tickets, which could lead to unauthorized entry. These issues highlighted the need for a more reliable and efficient ticket booking system.

Module – B : Online Ticket Booking Systems

With the development of computers and the internet, online ticket booking systems were introduced. These systems allowed users to book tickets through websites or mobile applications. Customers could view event details, check ticket availability, and book tickets from anywhere. This greatly improved convenience compared to manual systems.

Online ticketing systems also helped event organizers manage ticket sales more effectively. They could store ticket information in databases and generate reports easily. However, many early online systems still had limitations. Although tickets were booked online, users often had to print their tickets or show screenshots at the event. This did not completely eliminate the possibility of ticket duplication or misuse.

Another limitation was the lack of real-time ticket validation. Some systems did not update ticket status when a ticket was used. As a result, the same ticket could be used multiple times, leading to fraud. Additionally, some systems had poor user interfaces, making it difficult for users to navigate and complete the booking process.

Module – C : QR Code Based Ticketing Systems

To overcome the limitations of traditional and basic online systems, QR code-based ticketing systems were introduced. In these systems, each ticket is assigned a unique QR code. The QR code stores information related to the ticket, such as ticket ID and event details. At the event entrance, the QR code is scanned to verify the ticket.

QR code-based systems significantly improve the speed and accuracy of ticket verification. Scanning a QR code takes only a few seconds and reduces human error. It also helps prevent the use of fake or duplicate tickets. However, some QR-based systems do not connect to a centralized database for real-time verification. In such cases, QR codes can still be misused if they are copied.

The proposed Event Ticket Booking System improves this by combining QR code scanning with real-time database validation. When a QR code is scanned, the system checks the ticket details in the database and updates its status immediately. This ensures that each ticket can be used only once, making the system more secure and reliable.

CHAPTER - 3

SYSTEM STUDY FEASIBILITY STUDY

System study is an important phase in software development that involves understanding the existing system and analyzing the feasibility of the proposed system. It helps in deciding whether the new system can be developed successfully and whether it will be useful to users and organizations. In this project, the system study focuses on evaluating the feasibility of developing a web-based Event Ticket Booking System.

The current ticket booking process may involve either manual or based on simple online systems that lack proper security and real-time verification. Manual systems involve physical tickets and human checking, which is slow and error-prone. Basic online systems allow users to book tickets but often fail to prevent ticket duplication and misuse. These problems create the need for a more advanced system that supports digital tickets and secure verification.

The proposed Event Ticket Booking System aims to solve these issues by providing online ticket booking along with QR code-based ticket validation. To understand whether this system can be successfully implemented, a feasibility study is conducted. The feasibility study is divided into three main parts: technical feasibility, economic feasibility, and operational feasibility.

Technical Feasibility

Technical feasibility refers to whether the required technology and technical resources are available to develop the system. The Event Ticket Booking System is developed using modern and well-established technologies such as React for the frontend, Spring Boot for the backend, and MySQL for the database. These technologies are widely used in web application development and have strong community support.

React helps in building a responsive and interactive user interface, while Spring Boot provides a robust framework for developing secure and scalable backend services. MySQL is a reliable database system that efficiently stores and retrieves large amounts of data. Since these technologies are open-source and easily available, there are no major technical difficulties in developing the proposed system. Therefore, the system is technically feasible.

Economic Feasibility

Economic feasibility deals with the cost involved in developing and maintaining the system. The proposed Event Ticket Booking System is cost-effective because it uses open-source technologies such as Java, Spring Boot, React, and MySQL. These tools do not require any licensing fees, which reduces the overall development cost.

Additionally, the system reduces the need for manual ticket handling and physical ticket printing, which helps save money for event organizers. Digital tickets eliminate the cost of printing and distribution. Since the system is web-based, it can be hosted on low-cost servers, further reducing expenses.

Operational Feasibility

Operational feasibility refers to whether the system can be used effectively by its users. The proposed system is designed to be simple and user-friendly. Users can easily register, log in, view events, book tickets, and access their QR code tickets. Administrators can manage events and validate tickets through a simple interface.

The QR code scanning process is quick and easy, which improves the event entry experience. Since the system works through a web browser, users do not require any special training to use it. This makes the system operationally feasible.

Conclusion of Feasibility Study

From the above analysis, it is clear that the Event Ticket Booking System is technically, economically, and operationally feasible. The system can be developed using available technologies at low cost and can be easily used by users and administrators. Therefore, the proposed system is suitable for implementation and can effectively replace traditional ticket booking methods.

CHAPTER - 4

SYSTEM REQUIREMENTS SPECIFICATIONS

System Requirements Specification (SRS) is a detailed description of the system to be developed. It explains what the system should do and how it should perform. This chapter describes the existing system, the proposed system, and the hardware and software requirements of the Event Ticket Booking System.

4.1 Existing System

In the existing system, event ticket booking is mostly done either manually or through basic online platforms. In manual systems, customers have to visit ticket counters or agents to purchase tickets. This process is time-consuming and inconvenient, especially for popular events. It also leads to long queues and crowding at ticket counters.

Manual systems require physical tickets, which can be lost, damaged, or duplicated. There is no proper way to verify whether a ticket is genuine or already used. Event organizers must manually check each ticket at the entrance, which increases the chances of errors and unauthorized entry.

Some online ticket booking systems are available, but many of them do not provide proper real-time ticket validation. Although users can book tickets online, they still have to show printed tickets or screenshots, which can be misused. There is also a lack of centralized control and proper tracking of ticket usage. Due to these problems, the existing system is inefficient, insecure, and difficult to manage.

4.2 Proposed System

The proposed Event Ticket Booking System is a web-based solution designed to overcome the limitations of the existing system. It provides a fully digital platform for booking, managing, and validating event tickets. In this system, users can register and log in to their accounts, browse available events, and book tickets online. After booking, each user receives a digital ticket with a unique QR code. This QR code is used for ticket verification at the event entrance.

The administrator can add new events, update event details, manage ticket availability, and validate tickets by scanning QR codes. When a QR code is scanned, the system checks the ticket details in the database and updates the ticket status, ensuring that the same ticket cannot be used more than once.

The proposed system reduces manual work, increases security, and improves user convenience. It also provides a centralized platform for event organizers to manage all event and ticket information efficiently.

4.3 Hardware and Software Requirements

The successful implementation of the Event Ticket Booking System depends on proper hardware and software resources.

Hardware Requirements

- A computer or laptop
- Minimum 4 GB RAM
- Hard disk with at least 100 GB storage
- Internet connection

Software Requirements

- Operating System: Windows, Linux, or macOS
- Frontend: React
- Backend: Java with Spring Boot
- Database: MySQL
- Web Browser: Google Chrome, Edge, or Firefox
- Code Editor: VS Code or any IDE

CHAPTER - 5

SYSTEM ARCHITECTURE

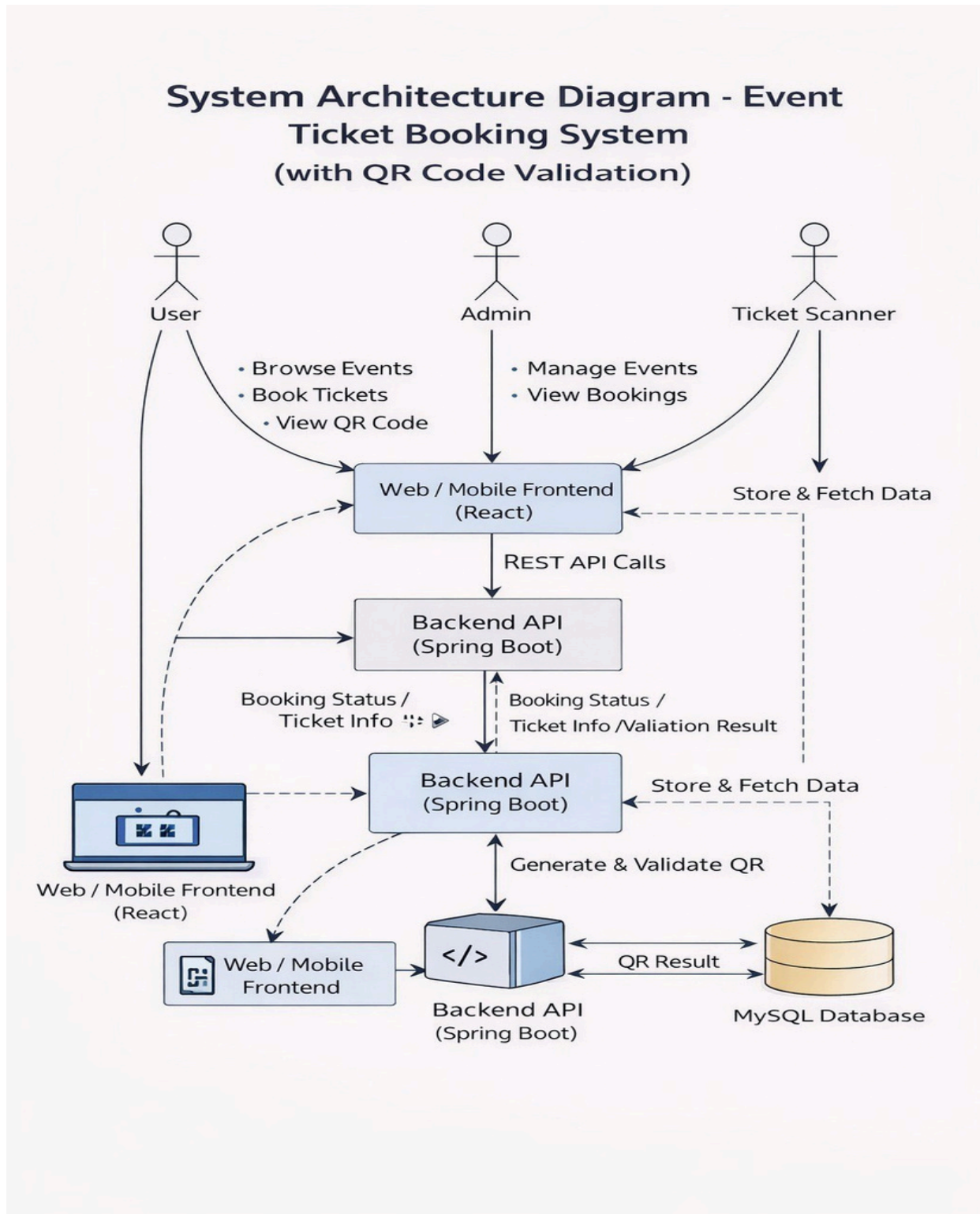


Fig 5.1 System Architecture

UML DIAGRAM'S:

UML (Unified Modeling Language) diagrams are used to represent the design and working of the Event Ticket Booking System in a visual and structured manner. UML diagrams help in understanding the functionality of the system, the relationship between different modules, and the flow of data from one component to another. In this project, UML diagrams are used to clearly describe how users and administrators interact with the system, how data is processed, and how different objects are connected.

The UML diagrams used in this project include Use Case Diagram, Class Diagram, Sequence Diagram, Collaboration Diagram, and Data Flow Diagram. Each diagram represents a different view of the system and helps in better understanding of the overall architecture.

The Use Case Diagram describes the interaction between actors and the system. In this project there are two main actors: User and Admin. The user can register, login, view events, book tickets, view booked tickets, and access the QR code. The admin can login, add or update events, view bookings, and validate tickets using QR code scanning. This diagram defines the major functionalities provided by the system.

The Class Diagram represents the static structure of the system. It shows the important classes such as User, Event, Ticket, Booking, QR Code, and Admin along with their attributes and relationships. This diagram explains how data is organized in the system and how one class is related to another, for example a User books a Ticket, a Ticket belongs to an Event, and Admin manages Events.

The Sequence Diagram explains the step-by-step flow of operations. When a user books a ticket, the request goes from the user interface to the backend, then to the database, and finally the QR code ticket is generated and returned to the user. This diagram helps to understand the order of message passing between frontend, backend, and database.

The Collaboration Diagram focuses on the interaction between different objects such as User, Event, Ticket, and Database. It shows how these components work together to complete tasks like ticket booking and ticket validation. It represents the communication links between objects.

The Data Flow Diagram (DFD) shows how data moves inside the system. User data, event data, and ticket data flow from the frontend to the backend and then to the database. During QR code scanning, the scanned data flows to the backend for verification and the result is returned to the admin. This ensures secure and real-time ticket validation.

Overall, UML diagrams play a very important role in this project by providing a clear model of the Event Ticket Booking System. They help developers, testers, and users to easily understand the system design, functionality, and data flow before actual implementation.

5.2 USE CASE DIAGRAM:

UML diagrams are used in the Event Ticket Booking System to represent the design and working of the application in a clear graphical way. These diagrams help to understand how the user and admin interact with the system, how data flows, and how different components are connected. The main UML diagrams used in this project are Use Case Diagram, Class Diagram, Sequence Diagram, Collaboration Diagram, and Data Flow Diagram.

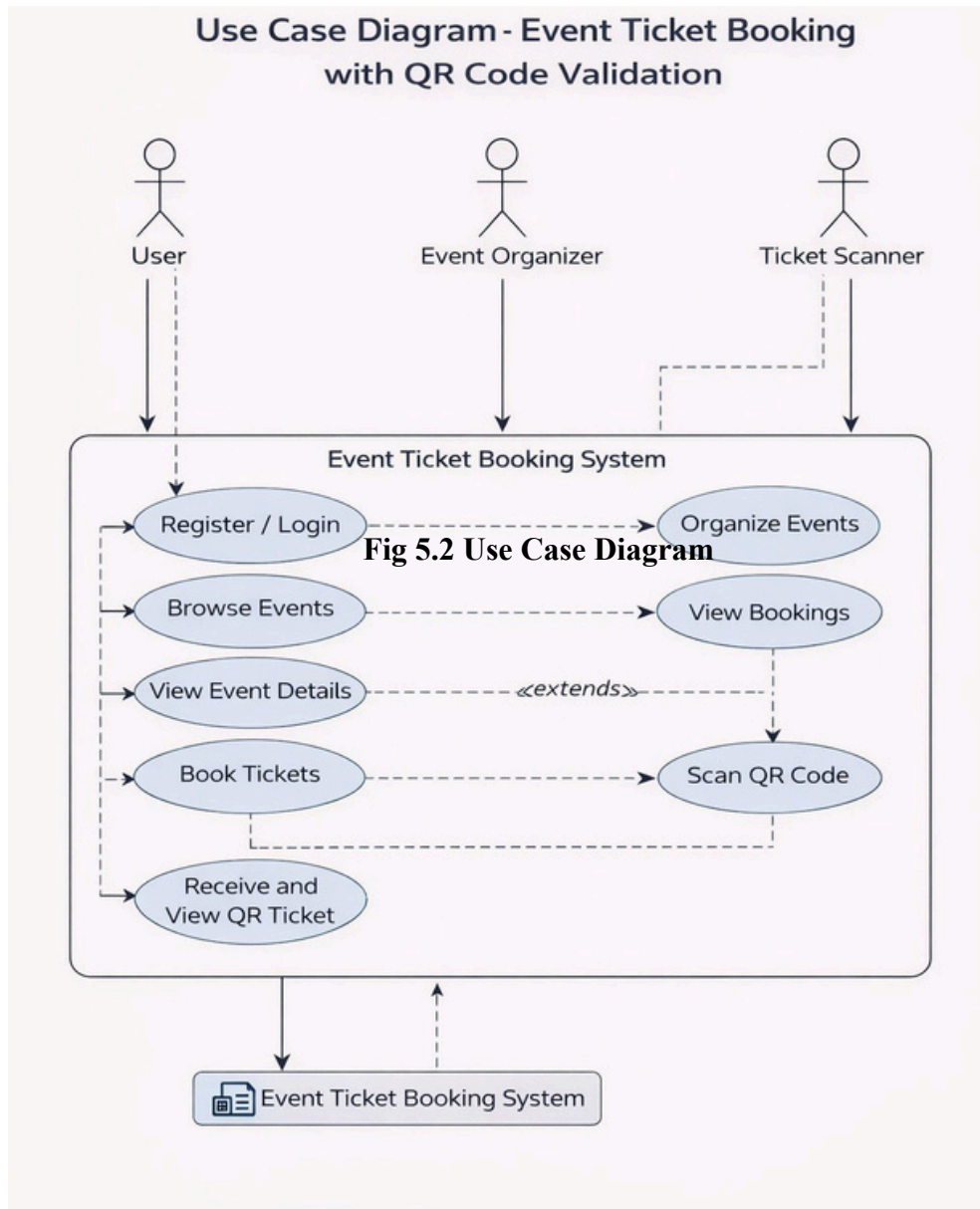


Fig 5.2 Use Case Diagram

5.3 CLASS DIAGRAM:

The Class Diagram of the Event Ticket Booking System represents the static structure of the system by showing the main classes, their attributes, operations, and relationships. This diagram explains how the data is organized in the system and how different modules are connected to each other.

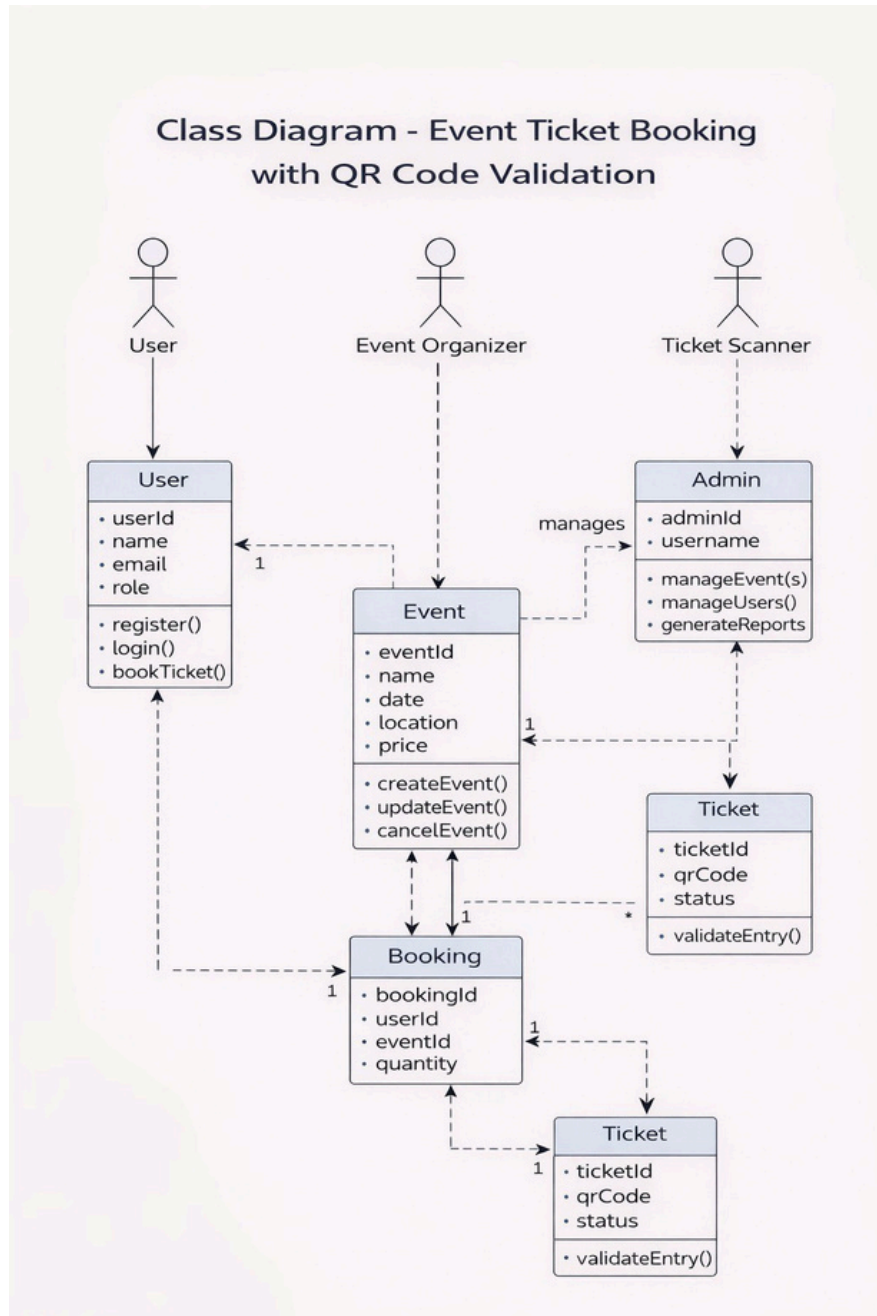


Fig 5.3 Class Diagram

5.4 SEQUENCE DIAGRAM:

The Sequence Diagram represents the dynamic behavior of the Event Ticket Booking System by showing the step-by-step interaction between different components in a time-ordered manner. It explains how messages are passed between the **User**, **Frontend**, **Backend**, **Database**, and **QR Code module** while performing major operations such as ticket booking and ticket validation.

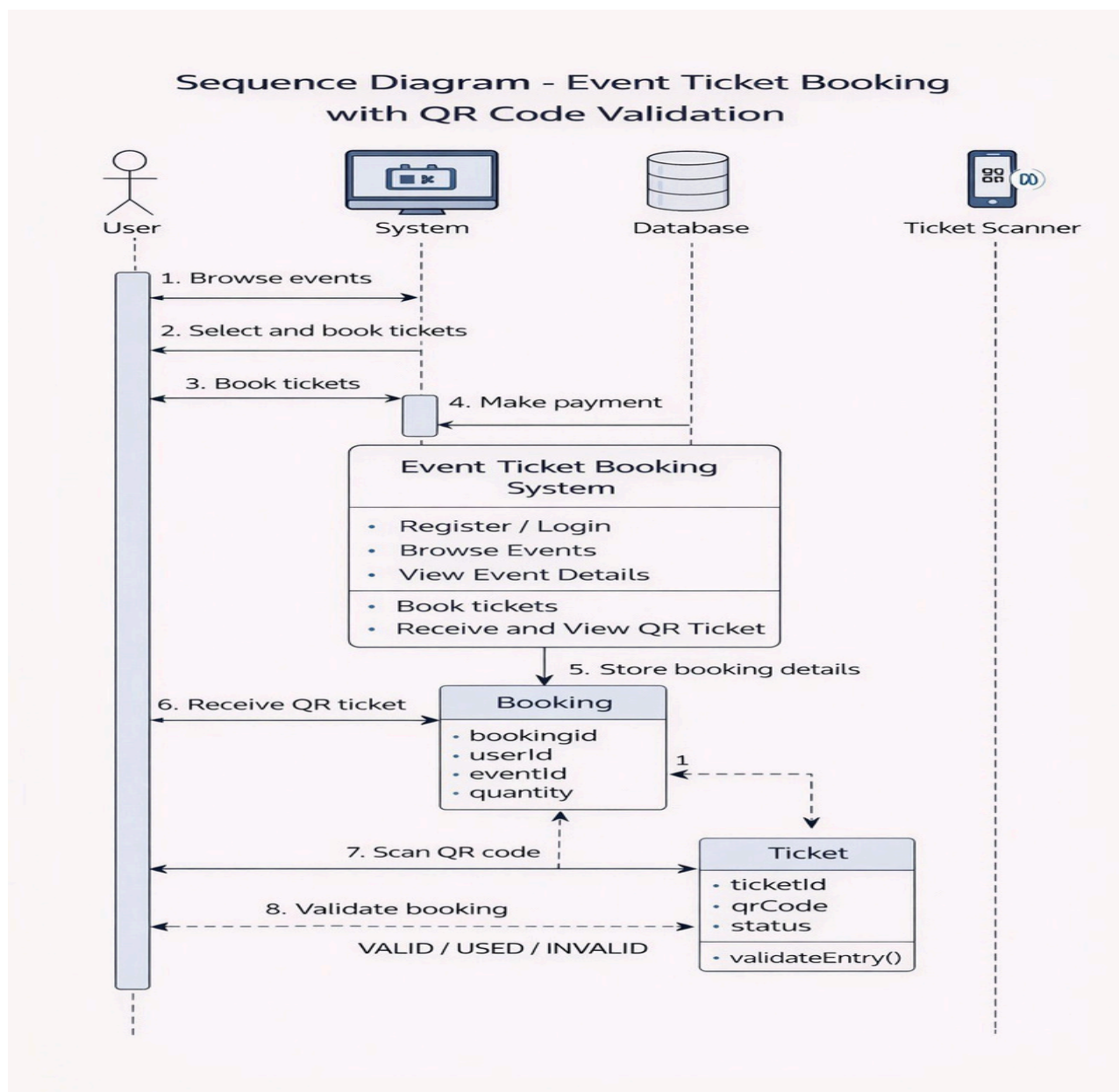


Fig 5.4 Sequence Diagram

5.5 COLLABRATION DIAGRAM:

The Collaboration Diagram represents the interaction between different objects in the Event Ticket Booking System. It shows how various components of the system work together to perform specific tasks such as ticket booking, event management, and QR code validation. Unlike the sequence diagram, the collaboration diagram focuses more on the relationships between objects rather than the time order of messages.

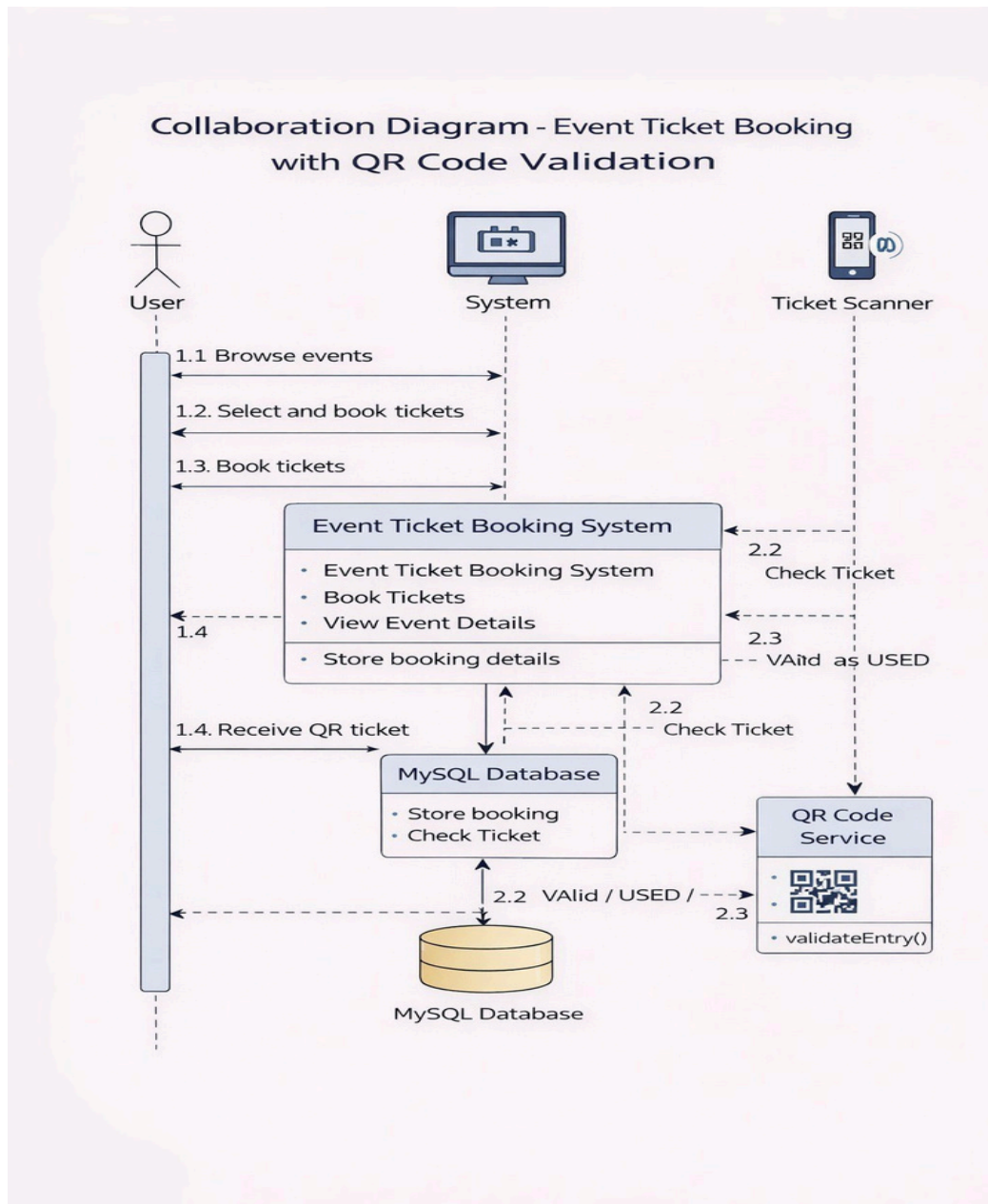
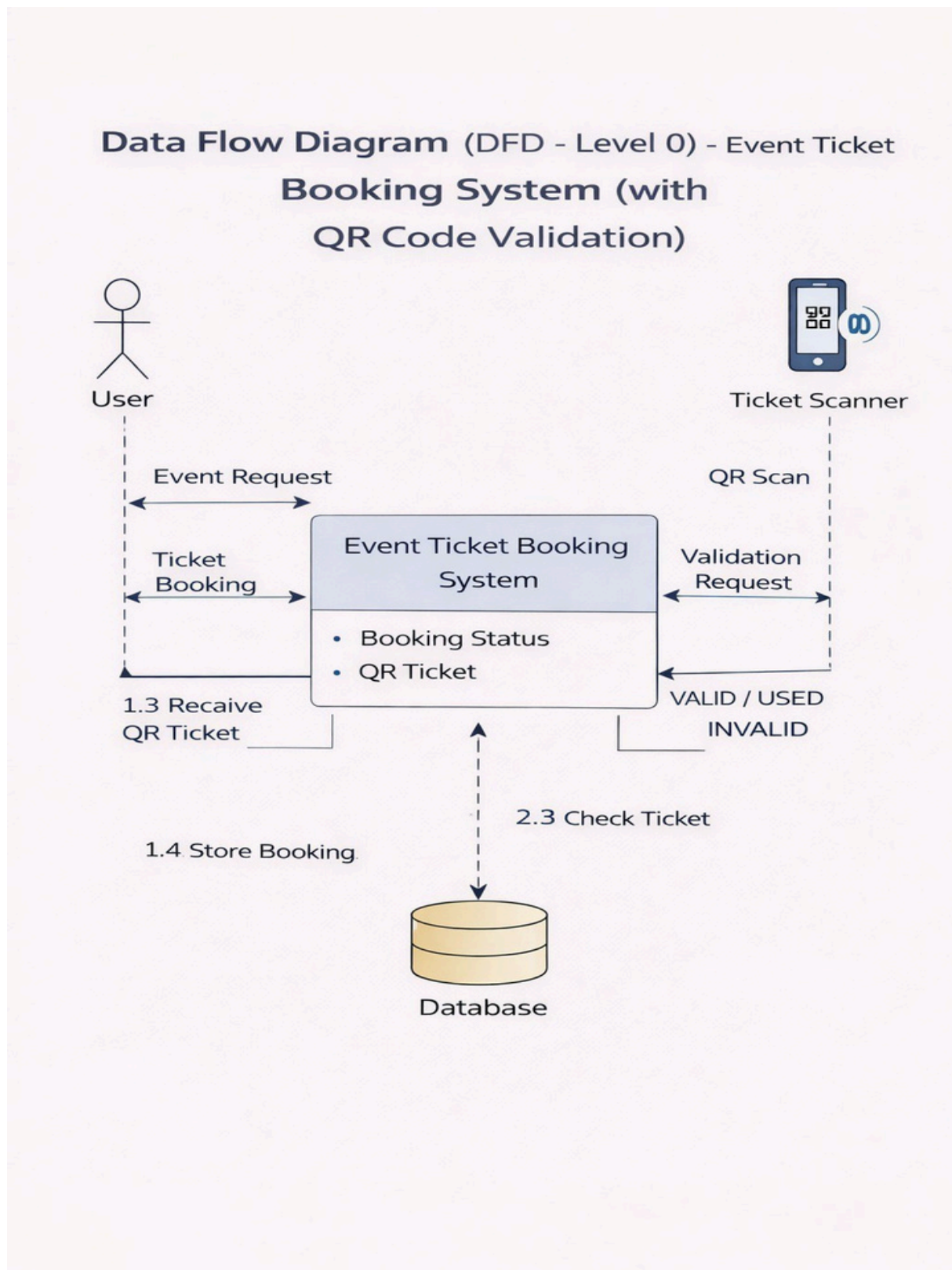
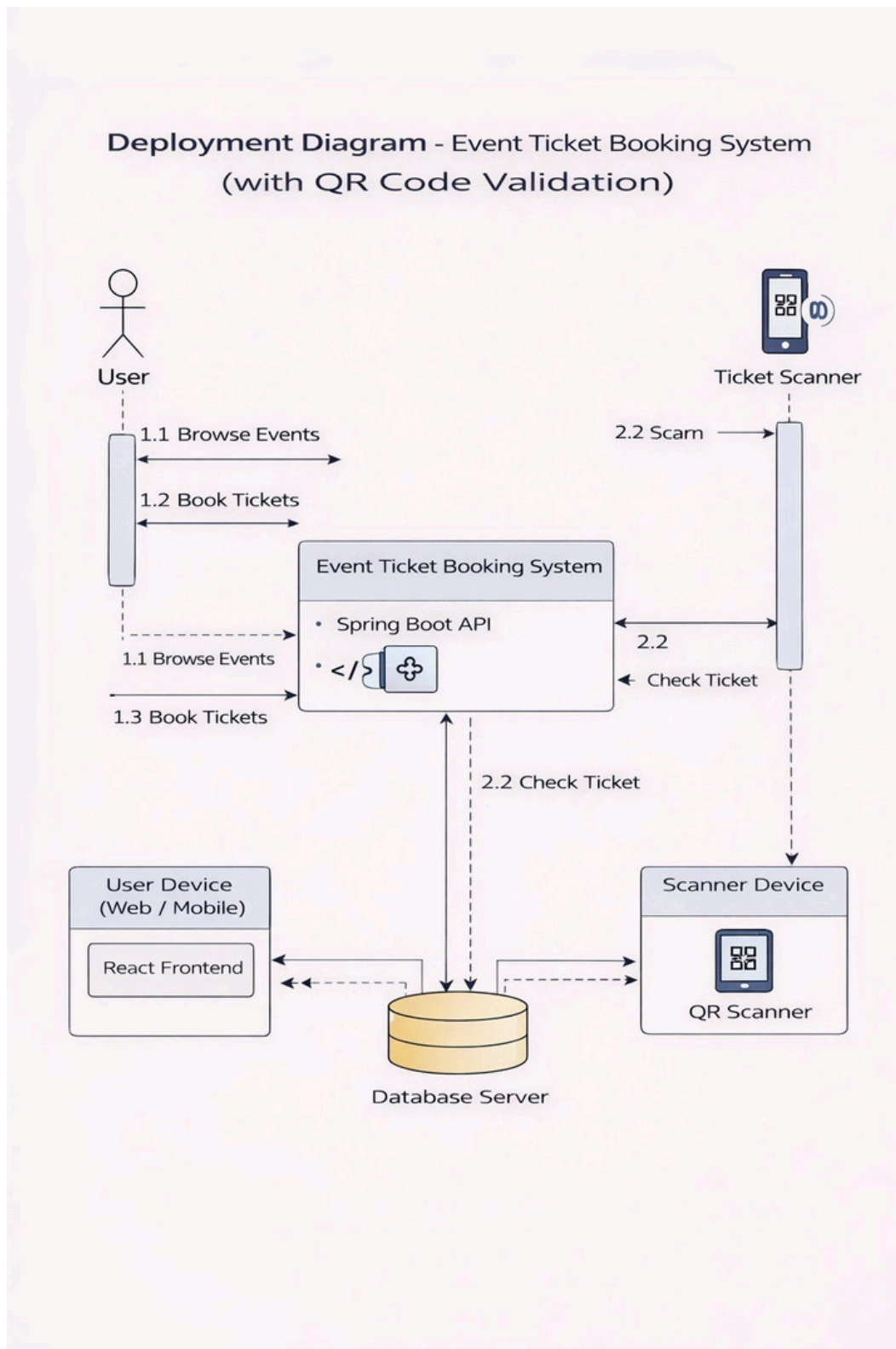


Fig 5.5 Collabracion Diagram

5.6 DATA FLOW DIAGRAM:**Fig 5.6 Data Flow Diagram**

5.7 DEPLOYMENT DIAGRAM:**Fig 5.7 Deployment Diagram**

CHAPTER – 6

IMPLEMENTATION

Implementation is the phase where the designed system is converted into a working application. In this stage, the proposed project “Event Ticket Booking with QR Code Validation” is developed using selected software tools and technologies. The main objective of implementation is to build a secure, reliable, and user-friendly system that fulfills all the requirements defined in earlier phases.

The system is implemented using a full-stack web development approach. The frontend is developed using React, the backend is developed using Spring Boot (Java), and the database is maintained using MySQL. The application follows a client–server architecture in which the frontend communicates with the backend through RESTful APIs.

6.1 MODULES

The project Event Ticket Booking with QR Code Validation is divided into three major modules:

- User Module
- Admin Module
- QR Code Validation Module

Module – A : User Module

This module is designed for customers who want to book tickets for events through the online platform.

Functions of User Module

- User Registration: New users can create an account by providing personal details such as name, email, mobile number, and password.
- User Login: Registered users can log in to the system using valid credentials.
- View Events: Users can browse available events with details like event name, date, venue, and ticket price.
- Book Tickets: Users can select an event and book the required number of tickets.
- View Booked Tickets: All booked tickets are displayed in the user dashboard.
- QR Code Access: After successful booking, a digital ticket with a unique QR code is generated.

Module – B : Admin Module

This module is used by the administrator to control and manage the entire system.

Functions of Admin Module

- Admin Login
- Add / Update / Delete Events
- View User Bookings
- Manage Ticket Details
- Validate Tickets using QR Scanner

The admin has full control over event management and ticket verification.

Module – C : QR Code Validation Module

This is the core module of the project Event Ticket Booking with QR Code Validation.

Functions

- Automatic QR code generation for every ticket
- Scanning QR code at event entry
- Verifying ticket details from database
- Updating ticket status after successful entry
- Preventing duplicate or fake tickets

CHAPTER - 7

SOFTWARE ENVIRONMENT

7.1 Introduction to Software Environment

The software environment defines the technologies, programming languages, frameworks, and tools used to develop the project Event Ticket Booking with QR Code Validation. This project is developed as a web-based application using modern full-stack technologies.

The environment supports frontend user interface development, backend business logic, database management, and QR code generation and validation. The selected environment ensures security, scalability, and easy maintenance of the system.

In this project, the software environment is used to integrate different modules such as user management, event management, ticket booking, and QR verification. All components work together through APIs and database connectivity to provide a smooth and automated ticketing process.

7.1.1 Operating Systems

The system can run on different operating systems such as Windows, Linux, and macOS. Since it is a web application, users only need a web browser and internet connection to access the system. The platform independence makes the application flexible and accessible from any device like laptop, desktop, or mobile.

Usage in our Project

- Developers can develop the application on any OS
- Server can be deployed on Linux or Windows
- Users can access system from PC
- Admin can validate tickets using any device

7.2 Java Programming Language

Java is used as the main programming language for backend development. It is an object-oriented and platform-independent language that supports application development. In this project, Java handles user authentication, ticket booking logic, event management, and QR code validation.

Usage in our Project

- Writing backend logic
- Creating REST APIs
- Connecting with MySQL database
- Implementing security and validations

7.2.1 Overview of Java

Java follows the concept of “Write Once, Run Anywhere.” Java programs are compiled into bytecode and executed on the Java Virtual Machine. It provides high security, exception handling, multithreading, and strong memory management which are essential for this project.

Usage

- Secure login and registration
- Handling multiple users at same time
- Exception handling during booking

7.2.2 Key Features of Java

Java provides important features such as object-oriented programming, platform independence, robustness, security, multithreading, and high performance. These features help in developing a reliable backend for event ticket booking and real-time QR verification.

Usage

- Secure data processing
- Reliable ticket generation
- Multi-user environment handling

7.3 Spring Boot Framework

Spring Boot is the backend framework used in this project. It simplifies development of web services and reduces configuration work. It provides built-in server support and easy integration with databases.

Usage

- Creating API endpoints
- Handling requests from React
- Managing business logic

7.3.1 Introduction to Spring Boot

Spring Boot is a module of Spring Framework used to create stand-alone applications. It allows developers to create REST APIs quickly and manage business logic efficiently. It is widely used for developing secure web applications.

Usage

- Fast development of backend
- Built-in Tomcat server
- Easy deployment

7.3.2 Role of Spring Boot in the Project

In this project, Spring Boot is used to create APIs for user registration, login, event management, ticket booking, and QR code validation. It connects the React frontend with the MySQL database and processes all business logic.

Usage

- /login API
- /book Ticket API
- /validate QR API
- /events API

7.3.3 Spring MVC

Spring MVC follows Model-View-Controller architecture. It separates user interface, business logic, and data layer. This improves code organization, security, and maintainability of the Event Ticket Booking system.

Usage

- Controller handles requests
- Service processes logic
- Repository handles DB

7.3.4 Advantages of Spring Boot

Spring Boot provides faster development, embedded server, easy database connectivity, security features, and scalability. These advantages make it suitable for this project.

Usage

- Secure authentication
- Easy maintenance
- Scalable for many events

7.4 Frontend Technologies

Frontend technologies are used to design the user interface through which users and admin interact with the system.

Usage

- Booking UI
- Admin dashboard
- QR display page

7.4.1 React Framework

React is used to build interactive pages such as login, registration, event list, ticket booking, my tickets, and admin dashboard. React allows fast rendering and component-based development.

Usage

- Dynamic event loading
- Form validation
- API integration

7.4.2 TypeScript and TSX

TypeScript is used with React to provide type safety and better code structure. TSX allows writing HTML-like code inside React components which makes UI development easier.

Usage

- Error free coding
- Maintainable components

7.4.3 CSS for UI Design

CSS is used to design layouts, colors, buttons, and responsive screens. It makes the application attractive and user-friendly on both mobile and desktop.

Usage

- Responsive design
- User friendly pages

7.4.4 Advantages of React Framework

React provides reusable components, fast performance, easy debugging, single page application support, and better user experience which are helpful for this project.

Usage

- Fast page loading
- Better UX for booking

7.5 Backend Technologies

Backend technologies handle server-side processing and database communication.

Usage

- Ticket logic
- QR verification
- Data storage

7.5.1 RESTful Web Services

REST APIs are used for communication between React frontend and Spring Boot backend. All operations like booking ticket and QR validation are performed through APIs.

Usage

- Send booking request
- Receive ticket data

7.5.2 Spring Data JPA

Spring Data JPA simplifies database operations such as insert, update, delete, and fetch without writing complex SQL queries.

Usage

- Save ticket
- Fetch events
- Update status

7.5.3 Hibernate ORM

Hibernate maps Java classes to database tables and manages data persistence efficiently in the system.

Usage

- Object to table mapping
- Automatic queries

7.6 Database Management System

Database is used to store all information related to users, events, tickets, and QR codes.

Usage

- Permanent data storage
- Retrieval for validation

\

7.6.1 MySQL Database

MySQL is used to store user details, event details, booking records, ticket information, and QR code data securely. It supports fast retrieval and large data handling.

Usage

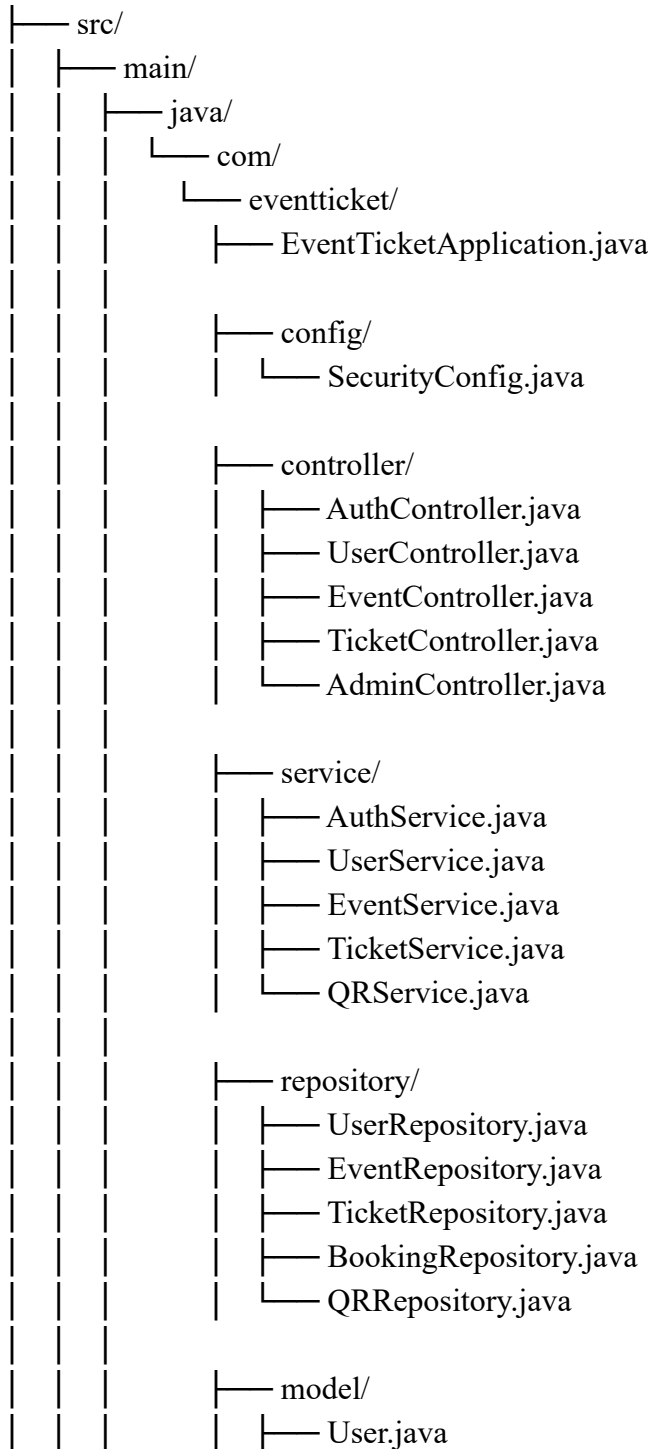
- User table
- Event table
- Ticket table
- Booking table
- QR table

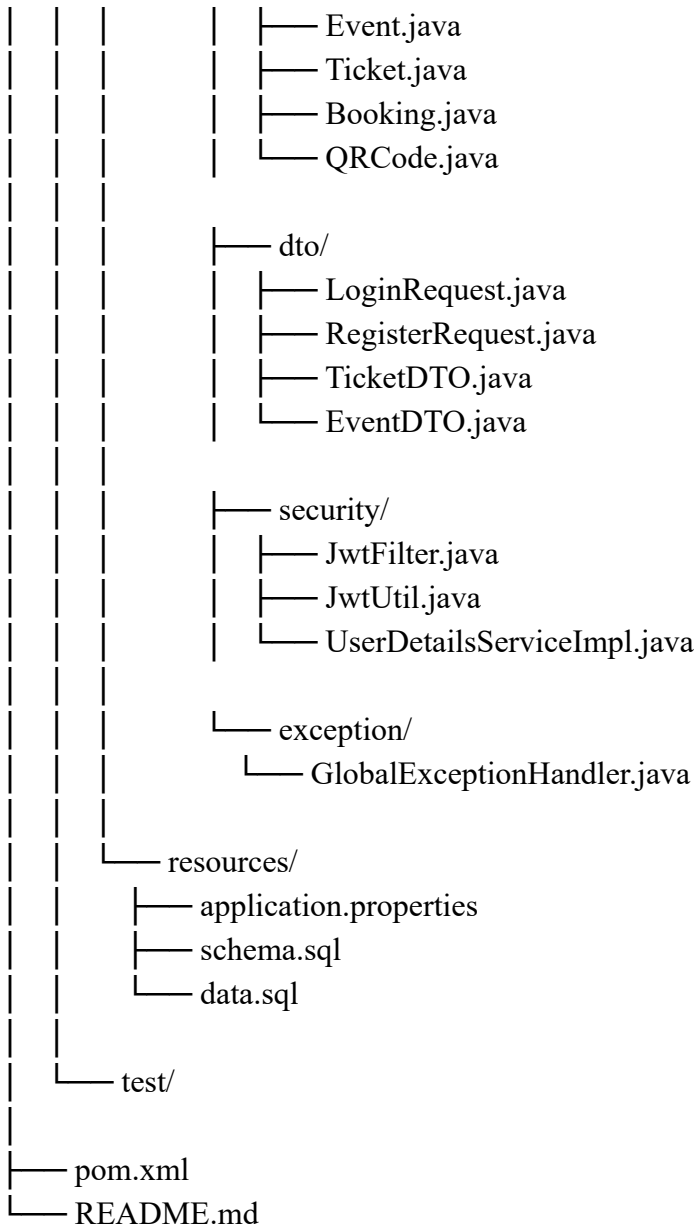
CHAPTER – 8

STRUCTURE

1. Backend (Spring Boot – Java 17): event-ticket-backend/

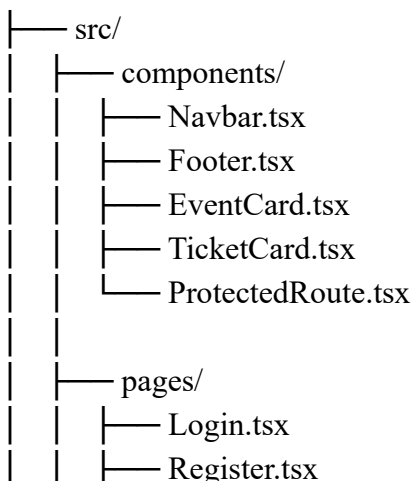
event-ticket-backend/





2. Frontend (React + TypeScript + TSX): event-ticket-frontend/

event-ticket-frontend/



```
|
|
|   |— Dashboard.tsx
|   |— EventList.tsx
|   |— BookTicket.tsx
|   |— MyTickets.tsx
|   |— QRView.tsx
|   |— AdminPanel.tsx
|
|   |— services/
|       |— authService.ts
|       |— eventService.ts
|       |— ticketService.ts
|       |— qrService.ts
|
|   |— models/
|       |— User.ts
|       |— Event.ts
|       |— Ticket.ts
|       |— Booking.ts
|
|   |— utils/
|       |— api.ts
|
|   |— App.tsx
|   |— index.tsx
|
|— package.json
|— README.md
```

SOURCE CODE

FULL BACKEND CODE – SPRING BOOT

EventTicketApplication.java

```
package com.eventticket;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class EventTicketApplication {

    public static void main(String[] args) {
        SpringApplication.run(EventTicketApplication.class, args);
    }
}
```

MODEL CLASSES

User.java

```
package com.eventticket.model;

import javax.persistence.*;

@Entity
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    private String password;
    private String mobile;

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }
}
```

```
public String getName() {  
    return name;  
}  
  
public void setName(String name) {  
    this.name = name;  
}  
  
public String getEmail() {  
    return email;  
}  
  
public void setEmail(String email) {  
    this.email = email;  
}  
  
public String getPassword() {  
    return password;  
}  
  
public void setPassword(String password) {  
    this.password = password;  
}  
  
public String getMobile() {  
    return mobile;  
}  
  
public void setMobile(String mobile) {  
    this.mobile = mobile;  
}  
}
```

Event.java

```
package com.eventticket.model;
```

```
import jakarta.persistence.*;
```

```
@Entity
```

```
public class Event {
```

```
    @Id
```

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
private Long id;  
private String eventName;  
private String date;  
private String venue;  
private double price;  
private int seats;
```

```
public Long getId() {  
    return id;  
}
```

```
public void setId(Long id) {  
    this.id = id;  
}
```

```
public String getEventName() {  
    return eventName;  
}
```

```
public void setEventName(String eventName) {  
    this.eventName = eventName;  
}
```

```
public String getDate() {  
    return date;  
}
```

```
public void setDate(String date) {  
    this.date = date;  
}
```

```
public String getVenue() {  
    return venue;  
}
```

```
public void setVenue(String venue) {  
    this.venue = venue;  
}
```

```
public double getPrice() {  
    return price;  
}
```

```
public void setPrice(double price) {  
    this.price = price;  
}  
  
public int getSeats() {  
    return seats;  
}  
  
public void setSeats(int seats) {  
    this.seats = seats;  
}  
}
```

Ticket.java

```
package com.eventticket.model;
```

```
import jakarta.persistence.*;
```

```
@Entity
```

```
public class Ticket {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private Long userId;
```

```
    private Long eventId;
```

```
    private String qrCode;
```

```
    private String status;
```

```
    public Long getId() {
```

```
        return id;
```

```
    }
```

```
    public void setId(Long id) {
```

```
        this.id = id;
```

```
    }
```

```
    public Long getUserId() {
```

```
        return userId;
```

```
    }
```

```
    public void setUserId(Long userId) {
```

```
this.userId = userId;
    }

    public Long getEventId() {
        return eventId;
    }

    public void setEventId(Long eventId) {
        this.eventId = eventId;
    }

    public String getQrCode() {
        return qrCode;
    }

    public void setQrCode(String qrCode) {
        this.qrCode = qrCode;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }
}

public Long getUserId() {
    return userId;
}

public void setUserId(Long userId) {
    this.userId = userId;
}

public Long getEventId() {
    return eventId;
}

public void setEventId(Long eventId) {
    this.eventId = eventId;
}
```

```
public String getQrCode() {
    return qrCode;
}

public void setQrCode(String qrCode) {
    this.qrCode = qrCode;
}

public String getStatus() {
    return status;
}

public void setStatus(String status) {
    this.status = status;
}
```

REPOSITORIES

UserRepository.java

```
package com.eventticket.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.eventticket.model.User;

public interface UserRepository extends JpaRepository<User, Long> {
}
```

EventRepository.java

```
package com.eventticket.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.eventticket.model.Event;

public interface EventRepository extends JpaRepository<Event, Long> {
}
```

TicketRepository.java

```
package com.eventticket.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.eventticket.model.Ticket;
```



```
public interface TicketRepository extends JpaRepository<Ticket, Long> {  
}
```

SERVICES – QRService.java

```
package com.eventticket.service;
```

```
import org.springframework.stereotype.Service;  
import java.util.UUID;
```

```
@Service  
public class QRService {  
    public String generateQR() {  
        return UUID.randomUUID().toString();  
    }  
}
```

CONTROLLERS – FULL LOGIC

```
package com.eventticket.controller;
```

```
import org.springframework.web.bind.annotation.*;  
import org.springframework.beans.factory.annotation.Autowired;
```

```
import com.eventticket.model.User;  
import com.eventticket.repository.UserRepository;
```

```
@RestController  
@RequestMapping("/auth")  
@CrossOrigin  
public class AuthController {
```

```
    @Autowired  
    UserRepository repo;  
    @PostMapping("/register")  
    public User register(@RequestBody User u) {  
        return repo.save(u);  
    }
```

```
    @PostMapping("/login")  
    public String login(@RequestBody User u) {  
        User db = repo.findAll().stream()  
            .filter(x -> x.getEmail().equals(u.getEmail()))  
            && x.getPassword().equals(u.getPassword()))  
            .findFirst().orElse(null);
```

```
return db != null ? "SUCCESS" : "FAIL";
    }
}
```

EventController.java

```
package com.eventticket.controller;

import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;

import com.eventticket.model.Event;
import com.eventticket.repository.EventRepository;
import java.util.List;

@RestController
@RequestMapping("/event")
@CrossOrigin
public class EventController {

    @Autowired
    EventRepository repo;

    @PostMapping("/add")
    public Event add(@RequestBody Event e) {
        return repo.save(e);
    }

    @GetMapping("/all")
    public List<Event> all() {
        return repo.findAll();
    }
}
```

TicketController.java

```
package com.eventticket.controller;

import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;

import com.eventticket.model.Ticket;
import com.eventticket.repository.TicketRepository;
import com.eventticket.service.QRService;
```

```
@RestController
@RequestMapping("/ticket")
@CrossOrigin
public class TicketController {

    @Autowired
    TicketRepository repo;

    @Autowired
    QRService qr;

    @PostMapping("/book")
    public Ticket book(@RequestBody Ticket t) {

        t.setQrCode(qr.generateQR());
        t.setStatus("ACTIVE");
        return repo.save(t);
    }

    @GetMapping("/validate/{qr}")
    public String validate(@PathVariable String qr) {
        Ticket t = repo.findAll().stream()
            .filter(x -> x.getQrCode().equals(qr))
            .findFirst().orElse(null);

        if (t == null)
            return "INVALID";

        if (t.getStatus().equals("USED"))
            return "ALREADY USED";

        t.setStatus("USED");
        repo.save(t);

        return "VALID";
    }
}
```

application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/eventdb
spring.datasource.username=root
spring.datasource.password=root
```

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

```
server.port=8080
```

DATABASE SCRIPT

```
CREATE DATABASE eventdb;
```

```
CREATE TABLE user (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(50),  
  email VARCHAR(50),  
  password VARCHAR(50),  
  mobile VARCHAR(15)  
);
```

```
CREATE TABLE event (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  event_name VARCHAR(50),  
  date VARCHAR(20),  
  venue VARCHAR(50),  
  price DOUBLE,  
  seats INT  
);
```

```
CREATE TABLE ticket (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  user_id INT,  
  event_id INT,  
  qr_code VARCHAR(100),  
  status VARCHAR(20)  
);
```

2. FRONTEND FULL CODE – REACT TSX

API SERVICE – api.ts

```
export const API = "http://localhost:8080";  
  
export async function post(url: string, data: any) {  
  return fetch(API + url, {  
    method: "POST",  
    headers: { "Content-Type": "application/json" },  
    body: JSON.stringify(data)  
  }).then(x => x.json());  
}
```

```
export async function get(url: string) {  
  return fetch(API + url).then(x => x.json());  
}
```

LOGIN PAGE – Login.tsx

```
import { useState } from "react";  
import { post } from "../services/api";  
  
export default function Login() {  
  
  const [email, setEmail] = useState("");  
  const [password, setPassword] = useState("");  
  
  const login = async () => {  
    const res = await post("/auth/login", { email, password });  
    alert(res);  
  };  
  
  return (  
    <div>  
      <h2>Login</h2>  
  
      <input  
        placeholder="Email"  
        onChange={e => setEmail(e.target.value)}  
      />  
  
      <input  
        type="password"  
        placeholder="Password"  
        onChange={e => setPassword(e.target.value)}  
      />  
  
      <button onClick={login}>Login</button>  
    </div>  
  );  
}
```

REGISTER PAGE – Register.tsx

```
import { useState } from "react";  
import { post } from "../services/api";  
export default function Register() {
```

```
const [name, setName] = useState("");
const [email, setEmail] = useState("");
const [password, setPassword] = useState("");
const [mobile, setMobile] = useState("");

const register = async () => {
  await post("/auth/register", { name, email, password, mobile });
  alert("Registered Successfully");
};

return (
  <div>
    <h2>Register</h2>

    <input
      placeholder="Name"
      onChange={e => setName(e.target.value)}
    />

    <input
      placeholder="Email"
      onChange={e => setEmail(e.target.value)}
    />

    <input
      placeholder="Password"
      type="password"
      onChange={e => setPassword(e.target.value)}
    />

    <input
      placeholder="Mobile"
      onChange={e => setMobile(e.target.value)}
    />

    <button onClick={register}>Register</button>
  </div>
);
```

EVENT LIST – EventList.tsx

```
import { useEffect, useState } from "react";
import { get } from "../services/api";
```

```
export default function EventList() {

  const [events, setEvents] = useState<any[]>([]);

  useEffect(() => {
    get("/event/all").then(setEvents);
  }, []);

  return (
    <div>
      <h2>Events</h2>

      {events.map(e => (
        <div key={e.id}>
          <h3>{e.eventName}</h3>
          <p>{e.date}</p>
          <p>{e.venue}</p>
          <p>Price: {e.price}</p>
        </div>
      ))}
    </div>
  );
}
```

BOOK TICKET – BookTicket.tsx

```
import { post } from "../services/api";

export default function BookTicket() {

  const book = async () => {
    await post("/ticket/book", {
      userId: 1,
      eventId: 1
    });
    alert("Ticket Booked Successfully");
  };

  return (
    <div>
      <button onClick={book}>
        Book Ticket
      </button>
    </div>
  );
}
```

```
    </div>
  );
}
```

MY TICKETS – MyTickets.tsx

```
import { useEffect, useState } from "react";
import { get } from "../services/api";

export default function MyTickets() {

  const [tickets, setTickets] = useState<any[]>([]);

  useEffect(() => {
    get("/ticket/all").then(setTickets);
  }, []);

  return (
    <div>
      <h2>My Tickets</h2>

      {tickets.map(t => (
        <div key={t.id}>
          <p>ID: {t.id}</p>
          <p>Status: {t.status}</p>
          <p>QR: {t.qrCode}</p>
        </div>
      ))}
    </div>
  );
}

import { post } from "../services/api";

export default function BookTicket() {

  const book = async () => {
    await post("/ticket/book", {
      userId: 1,
      eventId: 1
    });
    alert("Ticket Booked Successfully");
  };
}
```



```
return (  
  <div>  
    <button onClick={book}>  
      Book Ticket  
    </button>  
  </div>  
)  
}
```

MY TICKETS – MyTickets.tsx

```
import { useEffect, useState } from "react";  
import { get } from "../services/api";  
  
export default function MyTickets() {  
  
  const [tickets, setTickets] = useState<any[]>([]);  
  
  useEffect(() => {  
    get("/ticket/all").then(setTickets);  
  }, []);  
  
  return (  
    <div>  
      <h2>My Tickets</h2>  
  
      {tickets.map(t => (  
        <div key={t.id}>  
          <p>Ticket ID: {t.id}</p>  
          <p>Status: {t.status}</p>  
          <p>QR: {t.qrCode}</p>  
        </div>  
      ))}  
    </div>  
  );  
}
```

ADMIN SCANNER – Scanner.tsx

```
import { useState } from "react";  
import { API } from "../services/api";  
  
export default function Scanner() {
```

```
const [qr, setQr] = useState("");

const validate = async () => {
  const res = await fetch(
    API + "/ticket/validate/" + qr
  ).then(r => r.text());

  alert(res);
};

return (
  <div>
    <h2>QR Validation</h2>

    <input
      placeholder="Enter QR Code"
      onChange={e => setQr(e.target.value)}
    />

    <button onClick={validate}>
      Validate Ticket
    </button>
  </div>
);
}
```

App.tsx

```
import Login from "./pages/Login";
import Register from "./pages/Register";
import EventList from "./pages/EventList";
import BookTicket from "./pages/BookTicket";
import Scanner from "./pages/Scanner";

export default function App() {
  return (
    <div>
      <h1>Event Ticket Booking</h1>

      <Login />
      <Register />
      <EventList />
    </div>
  );
}
```

```
);  
}  
    <BookTicket />  
    <Scanner />  
  </div>  
);  
}
```

index.tsx

```
import React from "react";  
import ReactDOM from "react-dom";  
import App from "./App";  
  
ReactDOM.render(  
  <App />,  
  document.getElementById("root")  
)
```

package.json

```
{  
  "name": "event-ticket-frontend",  
  "version": "1.0.0",  
  "dependencies": {  
    "react": "^18.0.0",  
    "react-dom": "^18.0.0",  
    "typescript": "^4.5.0"  
  }  
}
```

CHAPTER – 9

SYSTEM TEST

System testing is the process of evaluating the complete and integrated Event Ticket Booking with QR Code Validation system to ensure that it meets the specified requirements. Testing is performed to identify errors, verify functionality, and ensure that the system works correctly in all possible conditions. The main objective of system testing is to deliver a reliable, secure, and user-friendly application.

The testing process checks whether all modules such as user registration, login, event management, ticket booking, QR code generation, and QR validation are working as expected. Both functional and non-functional testing methods are applied.

9.1 Objectives of Testing

- To verify that the system meets user requirements
- To detect and correct errors
- To ensure data accuracy and security
- To check QR code validation process
- To evaluate system performance
- To ensure user-friendly operation

9.2 Types of Testing

1. Unit Testing

Unit testing is performed on individual modules such as:

- Registration module
- Login module
- Event module
- Ticket booking module
- QR validation module

Each module is tested separately to ensure correct output for valid and invalid inputs.

2. Integration Testing

Integration testing verifies the communication between frontend, backend, and database.

- React frontend with Spring Boot APIs
- Backend with MySQL database
- QR module with ticket module

Data flow between modules is checked to ensure smooth operation.

3. Functional Testing

Functional testing checks whether the system performs the required functions. Tested functions:

- User can register and login
- User can view events
- User can book tickets
- QR code is generated after booking
- Admin can validate QR code
- Used ticket cannot be reused

9.3 User Acceptance Testing

The system was tested by sample users:

- Easy to use
- Fast booking
- Quick QR verification
- Clear interface

Feedback was positive.

9.4 Test Results

All major modules worked successfully:

- Registration – Working
- Login – Working
- Event Display – Working
- Booking – Working
- QR Generation – Working
- QR Validation – Working

No critical errors found.

Conclusion

System testing proved that the project “Event Ticket Booking with QR Code Validation” meets all functional and non-functional requirements. The application works efficiently with accurate QR verification and secure ticket management. Hence the system is ready for real-time usage.

CHAPTER - 10

RESULTS AND OUTPUTS SCREENS

Home Page

This output screen displays all available events in the platform. Users can view event name, date, venue, and ticket price. The interface is developed using React for fast rendering. A search and filter option helps users find events easily. Each event has a “Book Now” button for ticket reservation. The page proves successful integration between frontend and backend APIs.

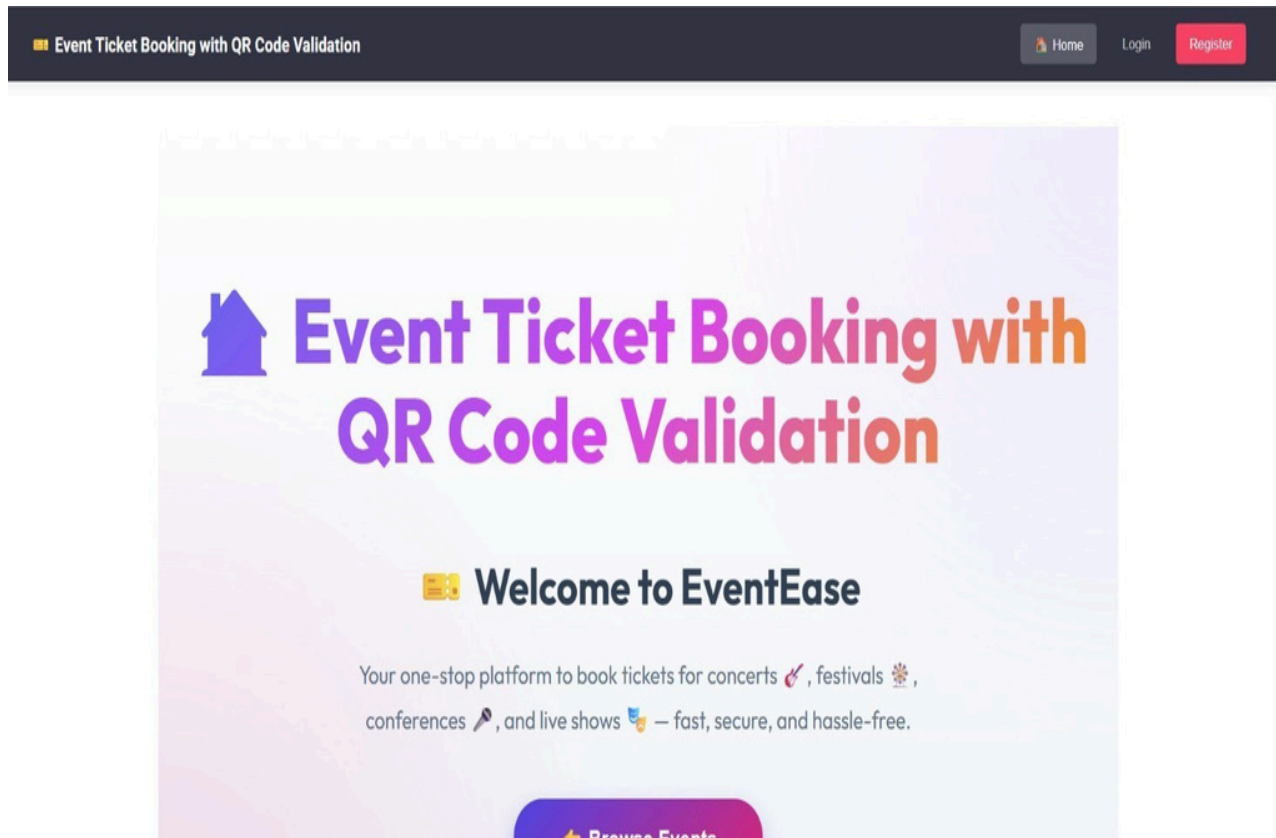


Fig 10.1 Home Page

User Login Page

This screen represents the login module of the Event Ticket Booking system. Registered users can enter their email and password to access the application. Proper validation is implemented to check incorrect credentials. The page provides options for new user registration and password recovery. Secure authentication is handled through the backend Spring Boot API. After successful login, the user is redirected to the dashboard. This screen ensures authorized access to the system.

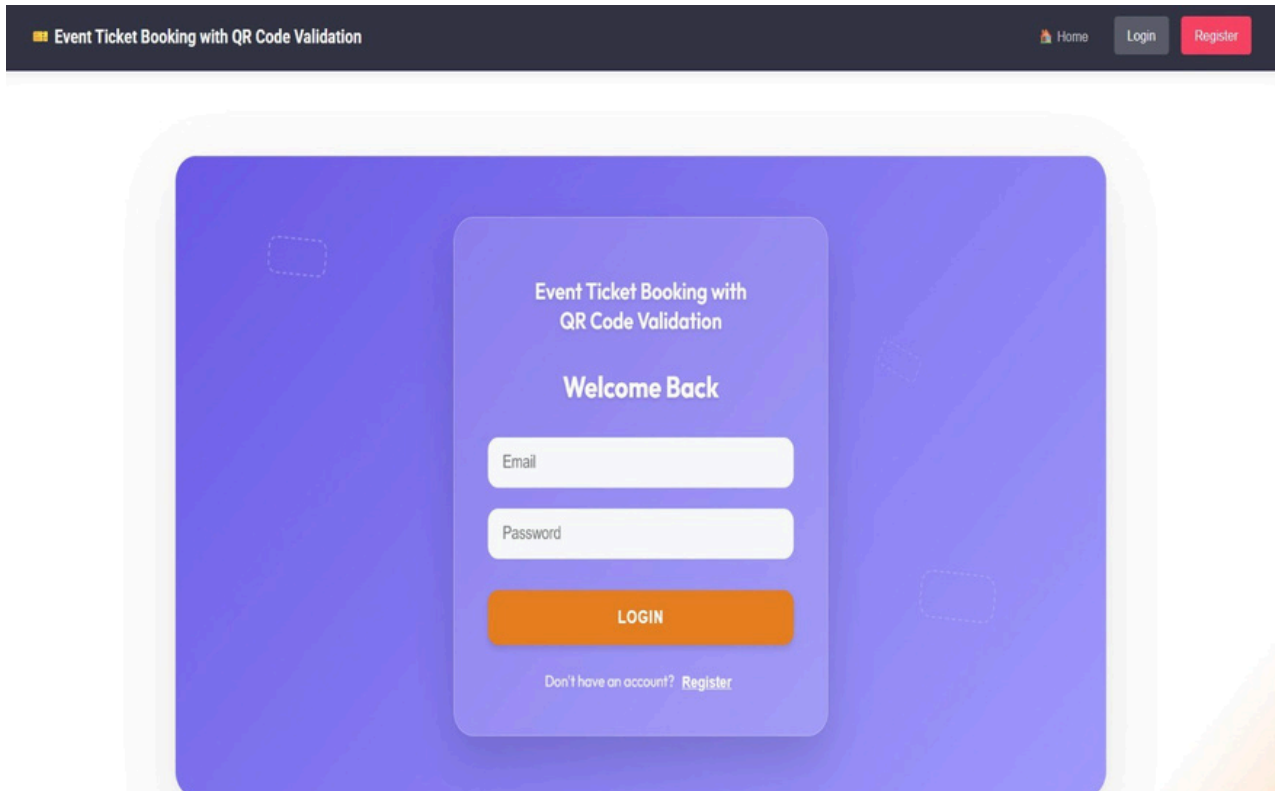


Fig 10.2 User Login Page

User Registration Page

The registration screen allows new users to create an account in the system. Users must provide details such as name, email, mobile number, and password. Form validations check empty fields and email format. Data is securely stored in the MySQL database. Duplicate email registration is restricted. After registration, the user can log in immediately. This module forms the entry point for all customers.

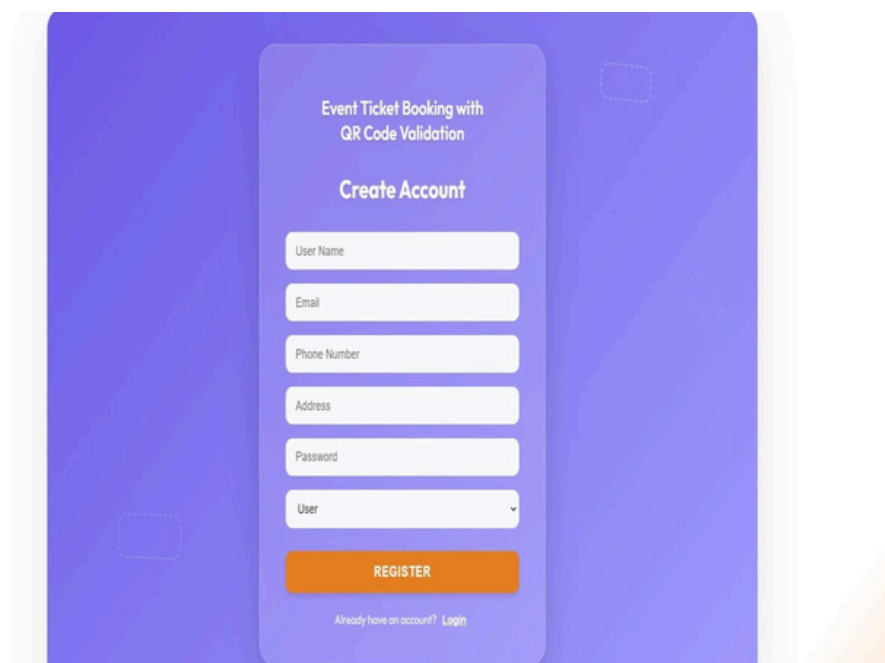


Fig 10.3 User Registration Page

Admin Dashboard

is the central control panel of the Event Ticket Booking system, providing complete management access to the administrator. It displays an overview of total users, events, bookings, and revenue in an organized and interactive layout. The dashboard allows the admin to create, update, and delete event details such as name, date, venue, and ticket price.

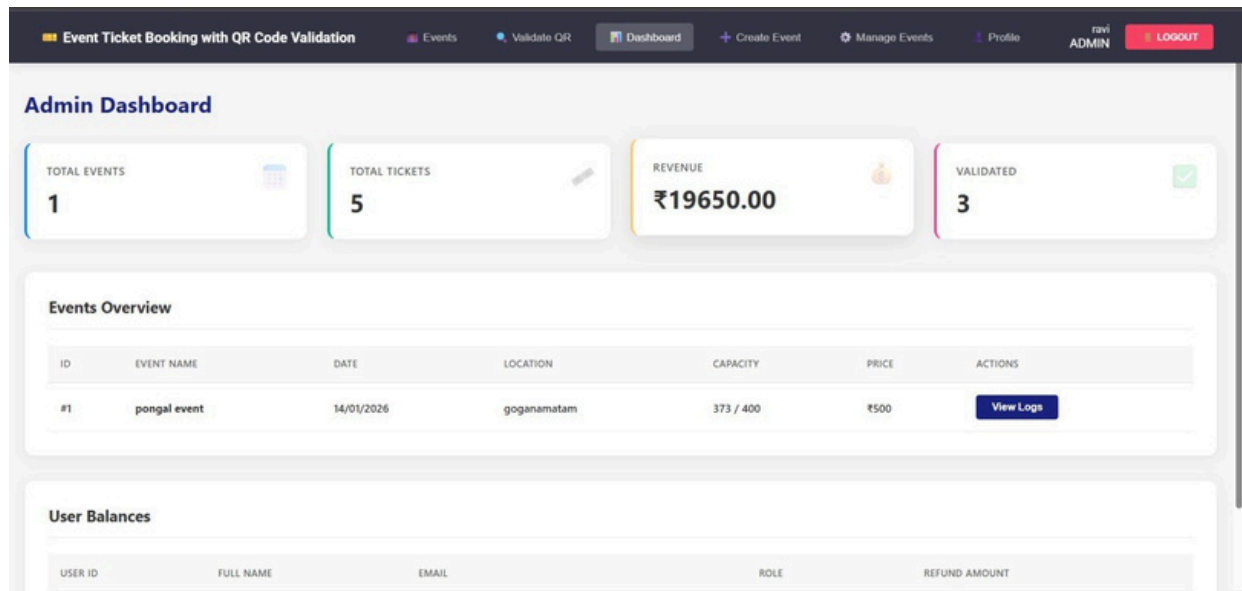


Fig 10.4 Admin Dashboard

Available Events

Users can browse multiple events in a structured and user-friendly interface. Each event card provides a “Book Now” option for quick ticket reservation. The system dynamically fetches event data from the backend database using REST API. The screen is designed using React to provide fast loading and smooth navigation. Users can view event descriptions before proceeding to booking confirmation.

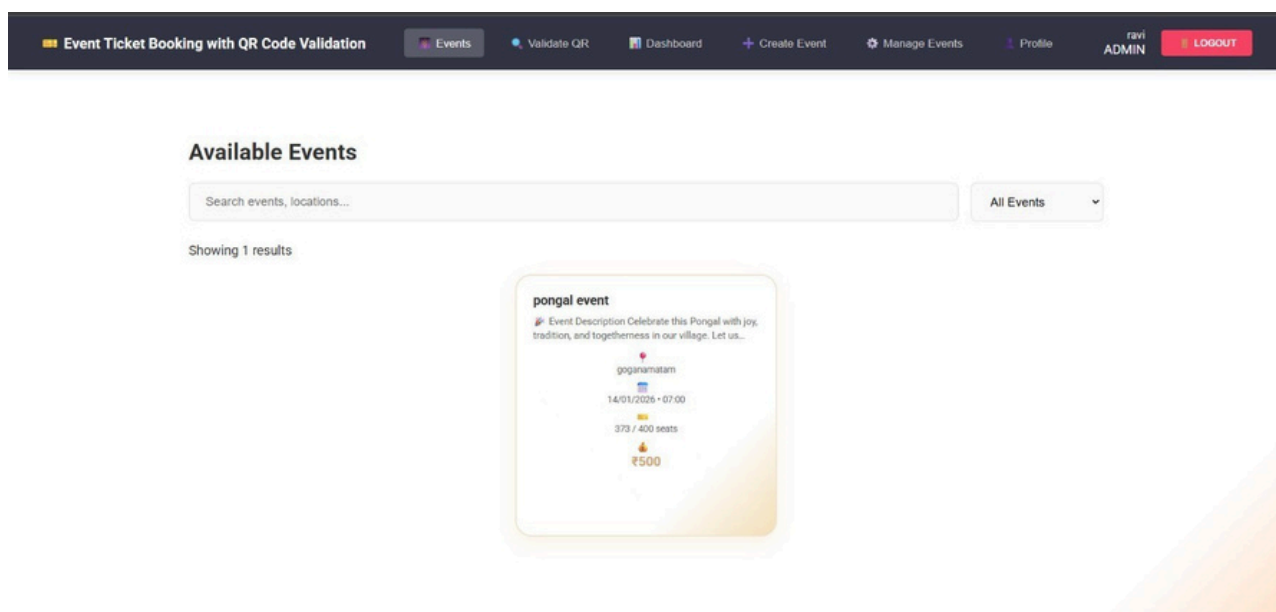


Fig 10.5 Available Events

Scan Qr Code

The form ensures proper input validation to avoid incorrect or incomplete data submission. Users can select the required number of tickets for the chosen event through this interface. The data entered by the user is securely sent to the backend using REST API services. React components are used to design an interactive and responsive booking form. Real-time validation messages guide the user to enter correct information.



Fig 10.6 Scan Qr Code

Create New Event

The Create Event screen allows the administrator to add new events into the system. It provides input fields to enter event details such as event name, date, time, venue, description, and ticket price. The form ensures that all mandatory fields are filled before submission to maintain data accuracy. Admin can also upload event images or banners to make the event more attractive to users. Validation mechanisms are implemented to prevent incorrect or incomplete data entry.

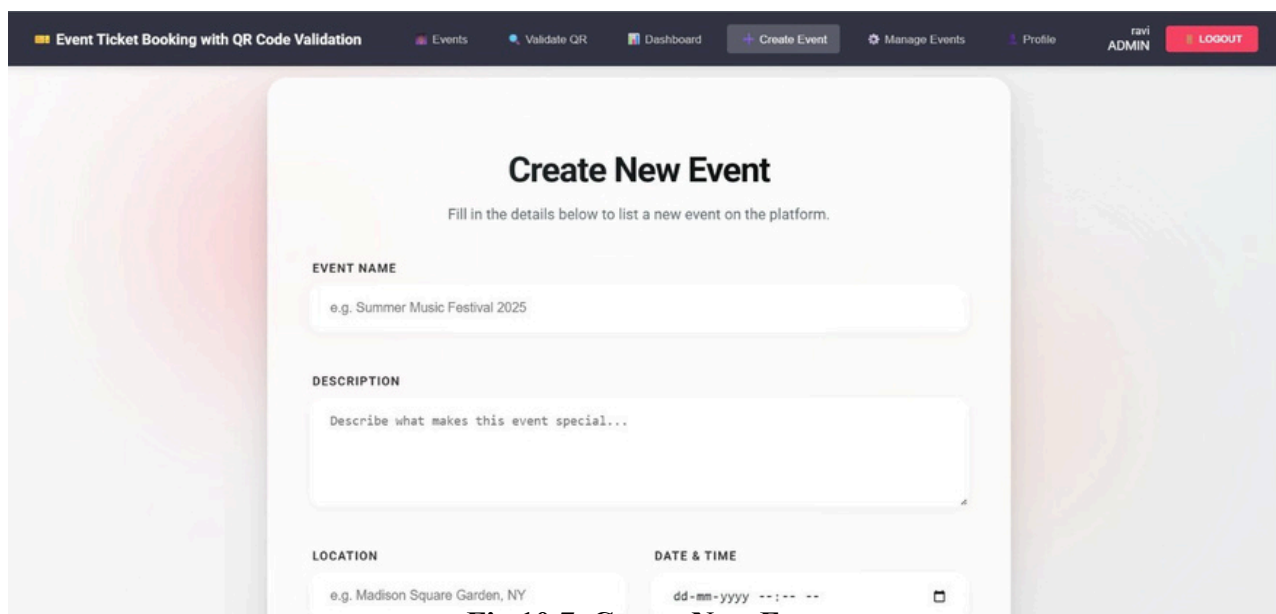


Fig 10.7 Create New Event

Event Management

The Event Management module is the core component of the “Event Ticket Booking with QR Code Validation” system. It enables the administrator to create, update, view, and delete event information dynamically. Each event contains details such as event name, description, venue, date, time, ticket price, and available seats. The system provides a user-friendly interface to manage multiple events without any technical complexity.

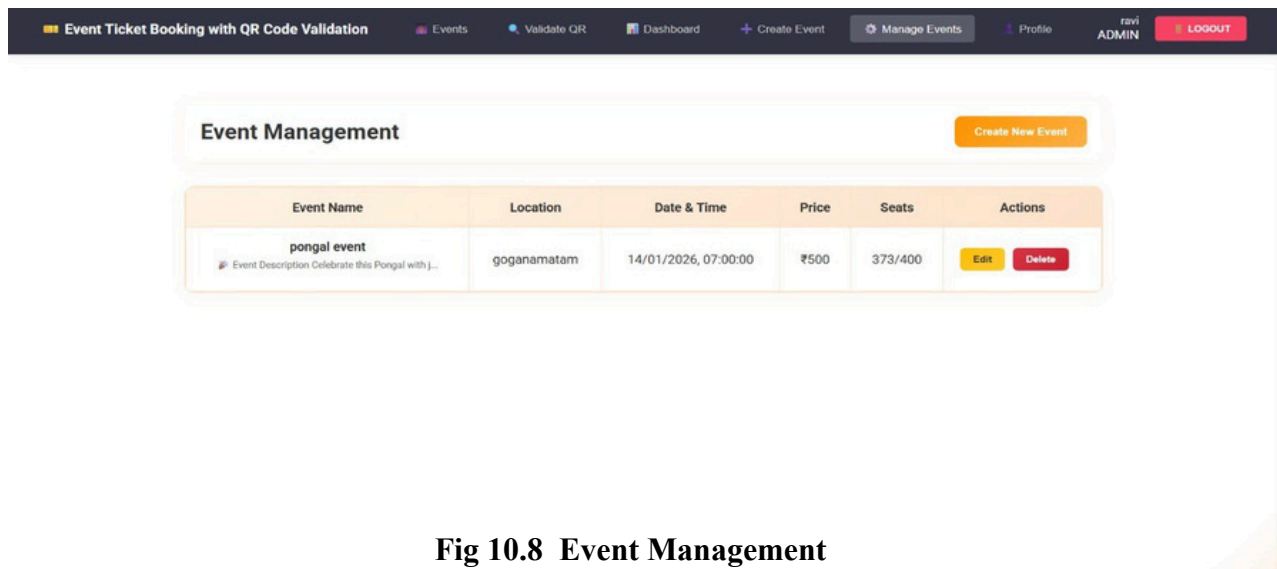


Fig 10.8 Event Management

Events Display Screen

The Events Display screen shows the detailed view of the selected event along with realtime seat availability. Users can view event information such as event name, venue, date, time, and ticket price before proceeding to seat selection. The seating layout is displayed graphically, indicating available, booked, and selected seats with different colors. This helps users to easily choose their preferred seats based on comfort and requirement.

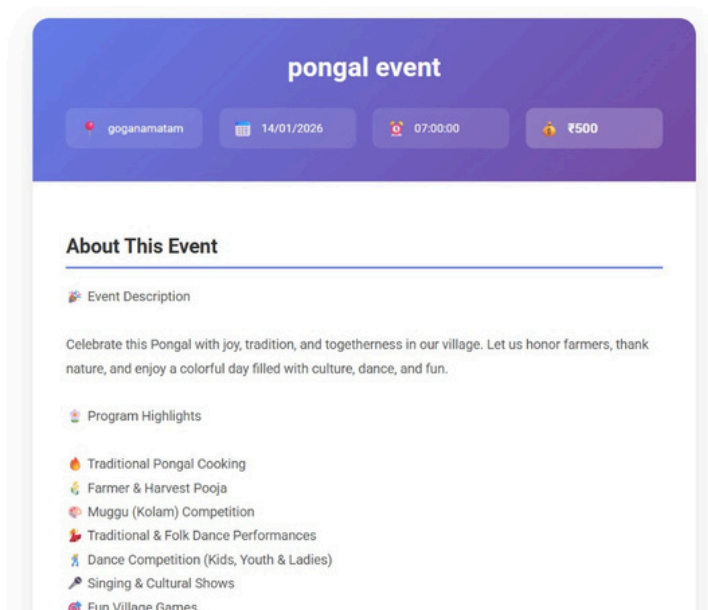


Fig 10.9 Events Display Screen

Seats

The Seats shows the detailed view of the selected event along with realtime seat availability. Users can view event information such as event name, venue, date, time, and ticket price before proceeding to seat selection. The seating layout is displayed graphically, indicating available, booked, and selected seats with different colors. This helps users to easily choose their preferred seats based on comfort and requirement.

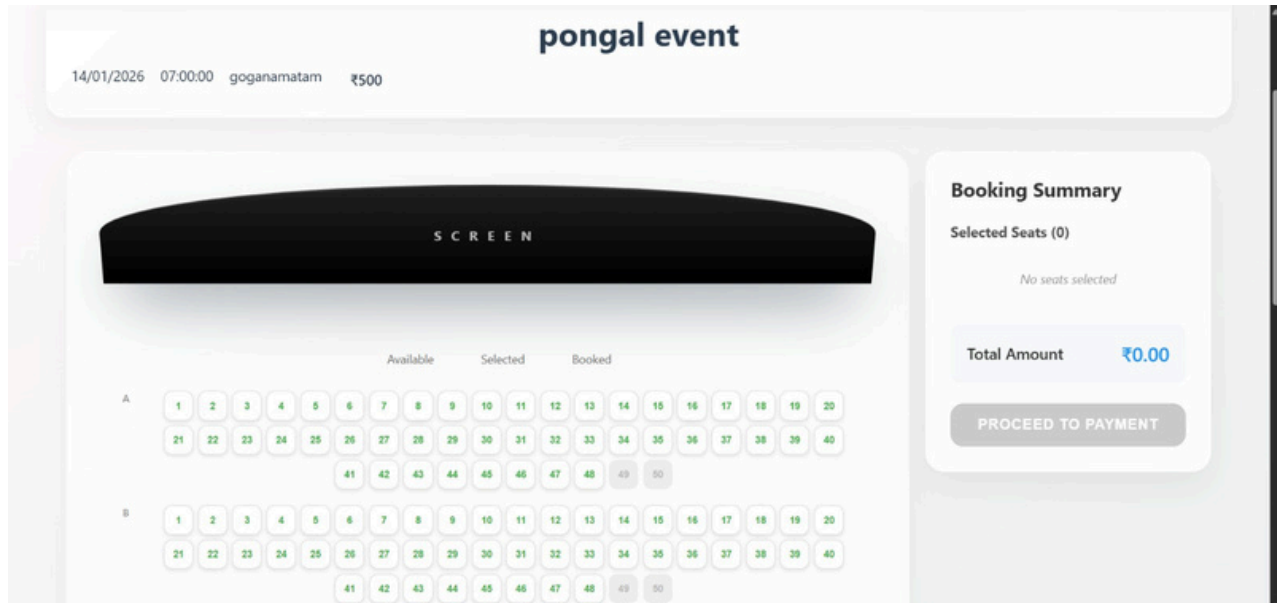


Fig 10.10 Seats

User and Admin Profile

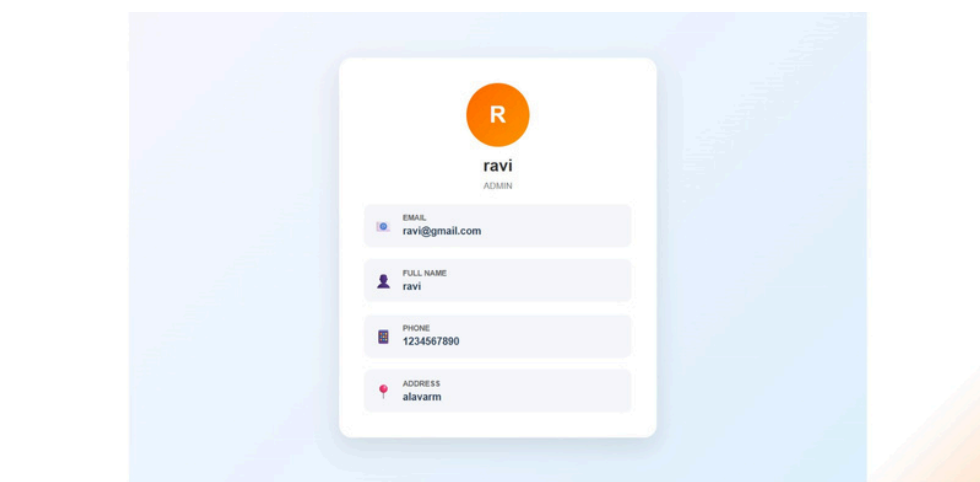


Fig 10.11 User and Admin Profile

Payment Section

The Payment Section enables users to complete ticket booking through a secure and user-friendly payment interface. After selecting the event and preferred seats, the total ticket amount is displayed along with booking details. The system supports multiple payment methods such as debit card, credit card, UPI, and net banking to provide flexibility to users. All payment transactions are processed through secure APIs to ensure data privacy and protection.

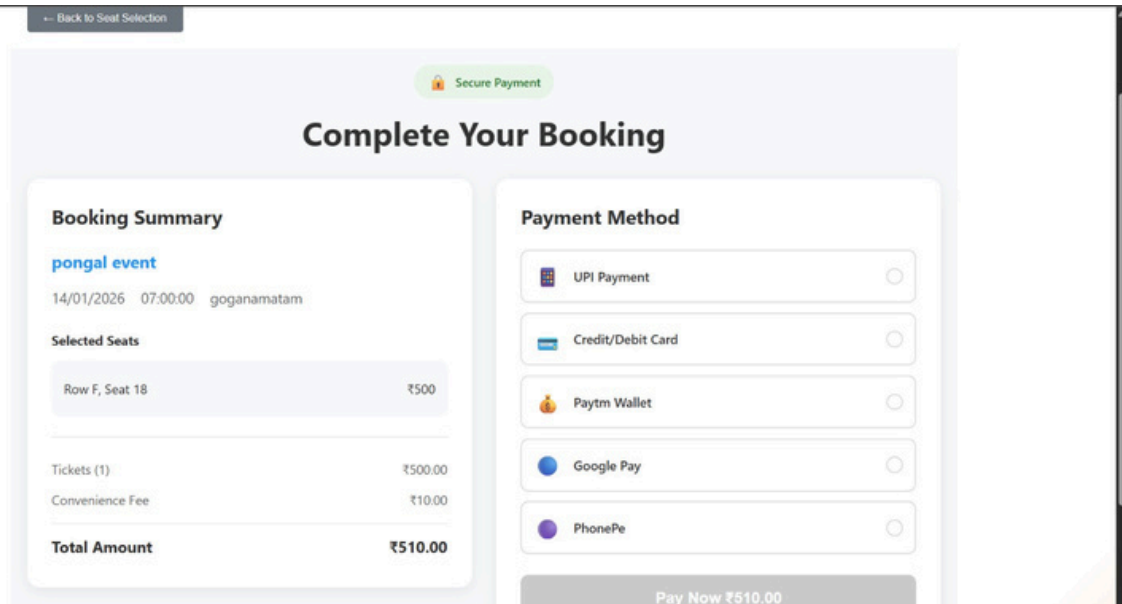


Fig 10.12 Payment Section

Tickets Section

The Tickets Section displays all booked tickets of the user in an organized manner. Each ticket contains important details such as event name, date, time, venue, seat numbers, booking ID, and QR code. Users can view, download, or share their tickets directly from this section. The QR code attached to each ticket is used for entry verification at the event venue. It improves user experience and supports paperless event entry through QR validation.

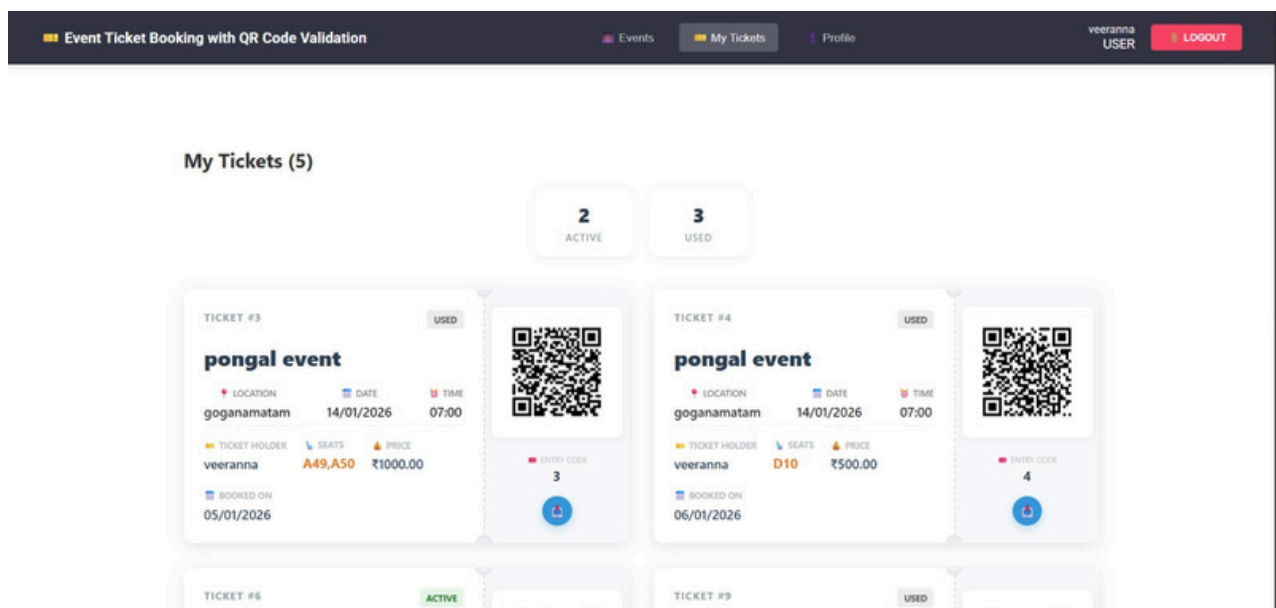


Fig 10.13 Tickets Section

CHAPTER - 11

CONCLUSION

The project was designed, developed, and tested to provide an efficient digital platform for managing events and ticketing operations. The system replaces the traditional manual ticket booking process with an automated solution, allowing users to browse events, book tickets online, and receive a unique QR code that acts as a digital entry pass. The integration of QR code technology ensures secure and reliable verification at the event venue.

The main objective of this project was to develop a system that reduces manual work, prevents ticket duplication, and provides real-time validation. These objectives have been achieved through the implementation of modern technologies such as React for frontend, Spring Boot for backend, and MySQL for database management. The system allows users to register, log in, view events, select seats, make payments, and access their ticket details from anywhere. The admin module provides complete control over event creation, ticket management, and QR validation.

One of the major advantages of this system is the QR code validation mechanism. Each ticket contains a unique QR code generated at the time of booking. At the event entrance, the QR code is scanned and verified with the database. If the ticket is valid, entry is granted; otherwise, the system rejects duplicate or fake tickets. This feature increases security and eliminates unauthorized access to events. It also reduces the need for physical tickets and manual checking.

This project is highly useful for colleges, seminar halls, movie theatres, cultural events, and conferences where large crowds need to be managed. It reduces long queues, saves time, and improves user experience. The paperless ticketing approach supports digital India initiatives and eco-friendly practices.

Although the current system meets all basic requirements, it can be further enhanced in the future by adding:

- Online payment gateway integration
- Seat layout graphical selection
- Advanced analytics for admin

In conclusion, the Event Ticket Booking with QR Code Validation project successfully fulfills its goals by providing a secure, reliable, and efficient solution for event management. The system proves that digital ticketing with QR technology is a practical and modern approach for real-time event entry management.

CHAPTER - 12

REFERENCES

- [1] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 8th ed., McGraw-Hill Education, 2015.
- [2] I. Sommerville, Software Engineering, 10th ed., Pearson Education, 2016.
- [3] Craig Walls, Spring Boot in Action, Manning Publications, 2016.
- [4] Joshua Bloch, Effective Java, 3rd ed., Addison-Wesley, 2018.
- [5] Mark Heckler, Spring Boot: Up and Running, O'Reilly Media, 2021.
- [6] Facebook Inc., React – A JavaScript Library for Building User Interfaces, Available: <https://react.dev>
- [7] Microsoft, TypeScript Documentation, Available: <https://www.typescriptlang.org/docs/>
- [8] Oracle Corporation, Java Platform, Standard Edition Documentation, Available: <https://docs.oracle.com/en/java/>
- [9] MySQL AB, MySQL Reference Manual, Available: <https://dev.mysql.com/doc/>
- [10] Hibernate Team, Hibernate ORM Documentation, Available: <https://hibernate.org>
- [11] Spring Framework Team, Spring Security Reference, Available: <https://spring.io/projects/spring-security>
- [12] Auth0, JSON Web Token (JWT) Introduction, Available: <https://jwt.io/introduction>
- [13] Apache Maven Project, Maven Documentation, Available: <https://maven.apache.org>
- [14] Roy Thomas Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Doctoral Dissertation, University of California, Irvine, 2000.
- [15] Mozilla Developer Network (MDN), Web Technologies Documentation, Available: <https://developer.mozilla.org>

- [16] Martin Fowler, Patterns of Enterprise Application Architecture, Addison-Wesley, 2003.
- [17] IEEE, IEEE Standard for Software Test Documentation, IEEE Std 829-2008.
- [18] W3C, HTML and CSS Specifications, Available: <https://www.w3.org>
- [19] Postman Inc., API Testing and Development, Available: <https://www.postman.com>
- [20] IntelliJ IDEA Documentation, JetBrains, Available: <https://www.jetbrains.com/idea/>
- [21] Sebastien Deleuze, Building RESTful Web Services with Spring Boot, Packt Publishing, 2018.
- [22] Alex Banks and Eve Porcello, Learning React: Functional Web Development with React and Redux, 2nd ed., O'Reilly Media, 2020.
- [23] Vlad Mihalcea, High-Performance Java Persistence, Vlad Mihalcea Publishing, 2016.
- [24] Gavin King, Hibernate in Action, Manning Publications, 2004.
- [25] Google Developers, Web Application Architecture Best Practices, Available: <https://developers.google.com>
- [26] Denso Wave Incorporated, QR Code Essentials and Standards, Available: <https://www.qrcode.com>
- [27] International Journal of Computer Applications, "QR Code Based Ticket Validation System," IJCA, 2022.
- [28] GeeksforGeeks, Spring Boot REST API Development Guide, Available: <https://www.geeksforgeeks.org>